A. a software interrupt
B. polling
C. an indirect jump
D. a privileged instruction 1

gate1999 operating-system normal ugcnetcse-dec2015-paper2 os-protection 2

Answer key

5.19.2 OS Protection: GATE CSE 2001 | Question: 1.13

A CPU has two modes -- privileged and non-privileged. In order to change the mode from privileged to non-privileged 3

A. a hardware interrupt is needed
B. a software interrupt is needed
C. a privileged instruction (which does not generate an interrupt) is needed
D. a non-privileged instruction (which does not generate an interrupt) is needed 4

gatecse-2001 operating-system normal os-protection 5

Answer key

5.19.3 OS Protection: GATE IT 2005 | Question: 19, UGCNET-June2012-III: 57

A user level process in Unix traps the signal sent on a Ctrl-C input, and has a signal handling routine that saves appropriate files before terminating the process. When a Ctrl-C input is given to this process, what is the mode in which the signal handling routine executes? 6 7

A. User mode
B. Kernel mode
C. Superuser mode
D. Privileged mode 8

gateit-2005 operating-system os-protection normal ugcnetcse-june2012-paper3 9

Answer key

5.20 Page Replacement (31)

Practice Tests: Test 1 (15Q) Test 2 (15Q) Test 3 (11Q)

5.20.1 Page Replacement: GATE CSE 1993 | Question: 21

The following page addresses, in the given sequence, were generated by a program: 10

$1\,2\,3\,4\,1\,3\,5\,2\,1\,5\,4\,3\,2\,3$

This program is run on a demand paged virtual memory system, with main memory size equal to 4 pages. Indicate the page references for which page faults occur for the following page replacement algorithms. 11

A. LRU
B. FIFO

Assume that the main memory is initially empty. 12

gate1993 operating-system page-replacement normal descriptive 13

Answer key

5.20.2 Page Replacement: GATE CSE 1994 | Question: 1.13

A memory page containing a heavily used variable that was initialized very early and is in constant use is removed then

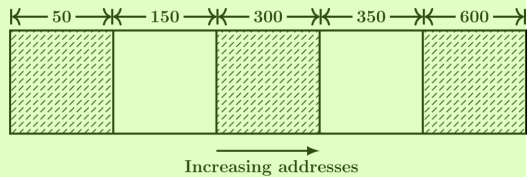
A. LRU page replacement algorithm is used
B. FIFO page replacement algorithm is used
C. LFU page replacement algorithm is used
D. None of the above

gate1994 operating-system page-replacement easy

Answer key

5.20.3 Page Replacement: GATE CSE 1994 | Question: 1.24

Consider the following heap (figure) in which blank regions are not in use and hatched region are in use.



1

The sequence of requests for blocks of sizes $300, 25, 125, 50$ can be satisfied if we use 2

A. either first fit or best fit policy (any one)
B. first fit but not best fit policy 3
C. best fit but not first fit policy
D. None of the above

gate1994 operating-system page-replacement normal 4

Answer key

5.20.4 Page Replacement: GATE CSE 1995 | Question: 1.8

Which of the following page replacement algorithms suffers from Belady's anamoly? 5

A. Optimal replacement B. LRU C. FIFO D. Both (A) and (C) 6

gate1995 operating-system page-replacement normal 7

Answer key

5.20.5 Page Replacement: GATE CSE 1995 | Question: 2.7

The address sequence generated by tracing a particular program executing in a pure demand based paging system with 100 records per page with 1 free main memory frame is recorded as follows. What is the number of page faults? 8

$0100, 0200, 0430, 0499, 0510, 0530, 0560, 0120, 0220, 0240, 0260, 0320, 0370$ 9

A. 13 B. 8 C. 7 D. 10 10

gate1995 operating-system page-replacement normal 11

Answer key

5.20.6 Page Replacement: GATE CSE 1997 | Question: 3.10, ISRO2008-57, ISRO2015-64

12

Dirty bit for a page in a page table 13

A. helps avoid unnecessary writes on a paging device
B. helps maintain LRU information 14
C. allows only read on a page
D. None of the above

gate1997 operating-system page-replacement easy isro2008 isro2015 15

Answer key

5.20.7 Page Replacement: GATE CSE 1997 | Question: 3.5

Locality of reference implies that the page reference being made by a process 16

A. will always be to the page used in the previous page reference 17
B. is likely to be to one of the pages used in the last few page references
C. will always be to one of the pages existing in memory
D. will always lead to a page fault

gate1997 operating-system page-replacement easy 18

Answer key

5.20.8 Page Replacement: GATE CSE 1997 | Question: 3.9

Thrashing

A. reduces page I/O B. decreases the degree of

C. implies excessive page I/O
multiprogramming
D. improve the system performance 1

gate1997 operating-system page-replacement easy

Answer key

5.20.9 Page Replacement: GATE CSE 2001 | Question: 1.21

Consider a virtual memory system with FIFO page replacement policy. For an arbitrary page access pattern, 2 increasing the number of page frames in main memory will

A. always decrease the number of page faults
B. always increase the number of page faults 3
C. sometimes increase the number of page faults
D. never affect the number of page faults

gatecse-2001 operating-system page-replacement normal 4

Answer key

5.20.10 Page Replacement: GATE CSE 2002 | Question: 1.23

The optimal page replacement algorithm will select the page that 5

A. Has not been used for the longest time in the past 6
B. Will not be used for the longest time in the future
C. Has been used least number of times
D. Has been used most number of times

gatecse-2002 operating-system page-replacement easy

Answer key

5.20.11 Page Replacement: GATE CSE 2004 | Question: 21, ISRO2007-44

The minimum number of page frames that must be allocated to a running process in a virtual memory 7 environment is determined by

A. the instruction set architecture
B. page size 8
C. number of processes in memory
D. physical memory size

gatecse-2004 operating-system virtual-memory page-replacement normal isro2007 9

Answer key 10

5.20.12 Page Replacement: GATE CSE 2005 | Question: 22, ISRO2015-36 11

Increasing the RAM of a computer typically improves performance because: 12

A. Virtual Memory increases
B. Larger RAMs are faster 13
C. Fewer page faults occur
D. Fewer segmentation faults occur

gatecse-2005 operating-system page-replacement easy isro2015 14

Answer key

5.20.13 Page Replacement: GATE CSE 2007 | Question: 56 15

A virtual memory system uses First In First Out (FIFO) page replacement policy and allocates a fixed 16 number of frames to a process. Consider the following statements:

P: Increasing the number of page frames allocated to a process sometimes increases the page fault rate.

Q: Some programs do not exhibit locality of reference.

Which one of the following is TRUE? 17

A. Both P and Q are true, and Q is the reason for P
B. Both P and Q are true, but Q is not the reason for P. 18
C. P is false but Q is true
D. Both P and Q are false.

gatecse-2007 operating-system page-replacement normal

Answer key

5.20.14 Page Replacement: GATE CSE 2007 | Question: 82

A process has been allocated 3 page frames. Assume that none of the pages of the process are available in the memory initially. The process makes the following sequence of page references (reference string): $\mathbf{1,2,1,3,7,4,5,6,3,1}$

If optimal page replacement policy is used, how many page faults occur for the above reference string?

A. 7 B. 8 C. 9 D. 10

gatecse-2007 operating-system page-replacement easy

Answer key

5.20.15 Page Replacement: GATE CSE 2007 | Question: 83

A process, has been allocated 3 page frames. Assume that none of the pages of the process are available in the memory initially. The process makes the following sequence of page references (reference string): $1,2,1,3,7,4,5,6,3,1$

Least Recently Used (LRU) page replacement policy is a practical approximation to optimal page replacement. For the above reference string, how many more page faults occur with LRU than with the optimal page replacement policy?

A. 0 B. 1 C. 2 D. 3

gatecse-2007 normal operating-system page-replacement

Answer key

5.20.16 Page Replacement: GATE CSE 2009 | Question: 9, ISRO2016-52

In which one of the following page replacement policies, Belady's anomaly may occur?

A. FIFO B. Optimal C. LRU D. MRU

gatecse-2009 operating-system page-replacement easy isro2016

Answer key

5.20.17 Page Replacement: GATE CSE 2010 | Question: 24

A system uses FIFO policy for system replacement. It has 4 page frames with no pages loaded to begin with. The system first accesses 100 distinct pages in some order and then accesses the same 100 pages but now in the reverse order. How many page faults will occur?

A. 196 B. 192 C. 197 D. 195

gatecse-2010 operating-system page-replacement normal

Answer key

5.20.18 Page Replacement: GATE CSE 2012 | Question: 42

Consider the virtual page reference string

$$1, 2, 3, 2, 4, 1, 3, 2, 4, 1$$

on a demand paged virtual memory system running on a computer system that has main memory size of 3 page frames which are initially empty. Let LRU, FIFO and $\mathrm{OPTIMAL}$ denote the number of page faults under the corresponding page replacement policy. Then

A. $\mathrm{OPTIMAL} < \mathrm{LRU} < \mathrm{FIFO}$
B. $\mathrm{OPTIMAL} < \mathrm{FIFO} < \mathrm{LRU}$
C. $\mathrm{OPTIMAL} = \mathrm{LRU}$
D. $\mathrm{OPTIMAL} = \mathrm{FIFO}$

gatecse-2012 operating-system page-replacement normal

Answer key

5.20.19 Page Replacement: GATE CSE 2014 | Set 1 | Question: 33

Assume that there are 3 page frames which are initially empty. If the page reference string is

$1, 2, 3, 4, 2, 1, 5, 3, 2, 4, 6$ the number of page faults using the optimal replacement policy is_____. 1

gatecse-2014-set1 operating-system page-replacement numerical-answers 2

Answer key

5.20.20 Page Replacement: GATE CSE 2014 | Set 2 | Question: 33

A computer has twenty physical page frames which contain pages numbered 101 through 120. Now a program accesses the pages numbered $1, 2, ..., 100$ in that order, and repeats the access sequence **THRICE**. Which one of the following page replacement policies experiences the same number of page faults as the optimal page replacement policy for this program? 3

A. Least-recently-used
B. First-in-first-out 4
C. Last-in-first-out
D. Most-recently-used

gatecse-2014-set2 operating-system page-replacement ambiguous 5

Answer key

5.20.21 Page Replacement: GATE CSE 2014 | Set 3 | Question: 20

6 7

A system uses 3 page frames for storing process pages in main memory. It uses the Least Recently Used (**LRU**) page replacement policy. Assume that all the page frames are initially empty. What is the total number of page faults that will occur while processing the page reference string given below?

$4, 7, 6, 1, 7, 6, 1, 2, 7, 2$

gatecse-2014-set3 operating-system page-replacement numerical-answers normal 8

Answer key

5.20.22 Page Replacement: GATE CSE 2015 | Set 1 | Question: 47

Consider a main memory with five-page frames and the following sequence of page references: $3, 8, 2, 3, 9, 1, 6, 3, 8, 9, 3, 6, 2, 1, 3$. Which one of the following is true with respect to page replacement policies First In First Out (FIFO) and Least Recently Used (LRU)? 9

A. Both incur the same number of page faults 10
B. FIFO incurs 2 more page faults than LRU
C. LRU incurs 2 more page faults than FIFO
D. FIFO incurs 1 more page faults than LRU

gatecse-2015-set1 operating-system page-replacement normal 11

Answer key

5.20.23 Page Replacement: GATE CSE 2016 | Set 1 | Question: 49

Consider a computer system with ten physical page frames. The system is provided with an access sequence $(a_1, a_2, \ldots, a_{20}, a_1, a_2, \ldots a_{20})$, where each a_i is a distinct virtual page number. The difference in the number of page faults between the last-in-first-out page replacement policy and the optimal page replacement policy is_____. 12

gatecse-2016-set1 operating-system page-replacement normal numerical-answers 13

Answer key

5.20.24 Page Replacement: GATE CSE 2016 | Set 2 | Question: 20

In which one of the following page replacement algorithms it is possible for the page fault rate to increase even when the number of allocated frames increases?

A. LRU (Least Recently Used)
B. OPT (Optimal Page Replacement)
C. MRU (Most Recently Used)
D. FIFO (First In First Out)

gatecse-2016-set2 operating-system page-replacement easy

Answer key

5.20.25 Page Replacement: GATE CSE 2017 | Set 1 | Question: 40

Recall that Belady's anomaly is that the page-fault rate may *increase* as the number of allocated frames increases. Now, consider the following statements: 1

- S_1: *Random page replacement* algorithm (where a page chosen at random is replaced) suffers from Belady's anomaly. 2
- S_2: *LRU page replacement* algorithm suffers from Belady's anomaly.

Which of the following is CORRECT? 3

A. S_1 is true, S_2 is true
B. S_1 is true, S_2 is false 4
C. S_1 is false, S_2 is true
D. S_1 is false, S_2 is false

gatecse-2017-set1 page-replacement operating-system normal 5

Answer key

5.20.26 Page Replacement: GATE CSE 2021 | Set 1 | Question: 11 6

In the context of operating systems, which of the following statements is/are correct with respect to paging? 7

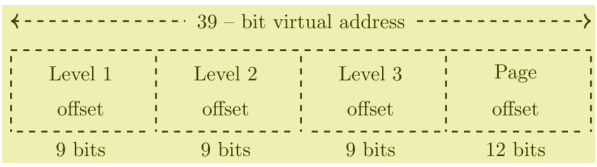
A. Paging helps solve the issue of external fragmentation
B. Page size has no impact on internal fragmentation
C. Paging incurs memory overheads
D. Multi-level paging is necessary to support pages of different sizes 8

gatecse-2021-set1 multiple-selects operating-system page-replacement one-mark 9

Answer key

5.20.27 Page Replacement: GATE CSE 2021 | Set 2 | Question: 48

Consider a three-level page table to translate a 39—bit virtual address to a physical address as shown below: 10



11

The page size is $4\,\text{KB}$ ($1\text{KB} = 2^{10}$ bytes) and page table entry size at every level is 8 bytes. A process P is currently using 2GB ($1\text{GB} = 2^{30}$ bytes) virtual memory which is mapped to 2GB of physical memory. The minimum amount of memory required for the page table of P across all levels is _____ KB. 12

gatecse-2021-set2 numerical-answers operating-system memory-management page-replacement two-marks 13

Answer key

5.20.28 Page Replacement: GATE CSE 2025 | Set 1 | Question: 44

In optimal page replacement algorithm, information about all future page references is available to the operating system (OS). A modification of the optimal page replacement algorithm is as follows:

The OS correctly predicts only up to next 4 page references (including the current page) at the time of allocating a frame to a page.

A process accesses the pages in the following order of page numbers:

$$1, 3, 2, 4, 2, 3, 1, 2, 4, 3, 1, 4$$

If the system has three memory frames that are initially empty, the number of page faults that will occur during execution of the process is _____. (Answer in integer)

gatecse2025-set1 operating-system page-replacement numerical-answers two-marks

Answer key

5.20.29 Page Replacement: GATE IT 2007 | Question: 12

The address sequence generated by tracing a particular program executing in a pure demand paging system with 100 bytes per page is 1

$$0100, 0200, 0430, 0499, 0510, 0530, 0560, 0120, 0220, 0240, 0260, 0320, 0410.$$

Suppose that the memory can store only one page and if x is the address which causes a page fault then the bytes from addresses x to $x + 99$ are loaded on to the memory. 2
How many page faults will occur?

A. 0 B. 4 C. 7 D. 8 3

gateit-2007 operating-system virtual-memory page-replacement normal 4

Answer key

5.20.30 Page Replacement: GATE IT 2007 | Question: 58

5 6

A demand paging system takes 100 time units to service a page fault and 300 time units to replace a dirty page. Memory access time is 1 time unit. The probability of a page fault is p. In case of a page fault, the probability of page being dirty is also p. It is observed that the average access time is 3 time units. Then the value of p is

A. 0.194 B. 0.233 C. 0.514 D. 0.981 7

gateit-2007 operating-system page-replacement probability normal 8

Answer key

5.20.31 Page Replacement: GATE IT 2008 | Question: 41

9 10

Assume that a main memory with only 4 pages, each of 16 bytes, is initially empty. The CPU generates the following sequence of virtual addresses and uses the Least Recently Used (LRU) page replacement policy.

$0, 4, 8, 20, 24, 36, 44, 12, 68, 72, 80, 84, 28, 32, 88, 92$

How many page faults does this sequence cause? What are the page numbers of the pages present in the main memory at the end of the sequence?

A. 6 and $1,2,3,4$ B. 7 and $1,2,4,5$ C. 8 and $1,2,4,5$ D. 9 and $1,2,3,5$ 11

gateit-2008 operating-system page-replacement normal 12

Answer key

5.21 Precedence Graph (3)

5.21.1 Precedence Graph: GATE CSE 1989 | Question: 11b

13

Consider the following precedence graph $(\text{Fig.}6)$ of processes where a node denotes a process and a directed edge from node P_i to node P_j implies; that P_i must complete before P_j commences. Implement the graph using FORK and JOIN constructs. The actual computation done by a process may be indicated by a comment line.

14

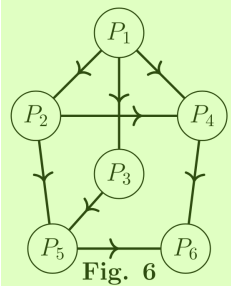


Fig. 6

gate1989 descriptive operating-system precedence-graph process-synchronization

Answer key

5.21.2 Precedence Graph: GATE CSE 1991 | Question: 01-xii

A given set of processes can be implemented by using only **parbegin**/**parend** statement, if the precedence graph of these processes is _____ 1

gate1991 operating-system normal precedence-graph fill-in-the-blanks 2

Answer key

5.21.3 Precedence Graph: GATE CSE 1992 | Question: 12-a

Draw the precedence graph for the concurrent program given below 3

```
S1
parbegin
   begin
      S2:S4
   end;
   begin
      S3;
      parbegin
         S5;
         begin
            S6:S8
         end
      parend
   end;
   S7
parend;
S9
```

4

gate1992 operating-system normal concurrency precedence-graph descriptive 5

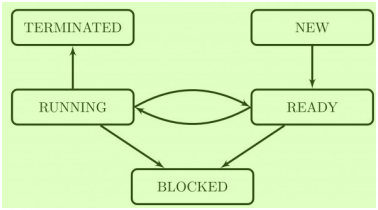
Answer key 6

5.22 Process (5)

Practice Test: Test 1 (7Q) 7

5.22.1 Process: GATE CSE 1996 | Question: 1.18

The process state transition diagram in the below figure is representative of 8



9

A. a batch operating system
B. an operating system with a preemptive scheduler
C. an operating system with a non-preemptive scheduler
D. a uni-programmed operating system

10

gate1996 operating-system normal process 11

Answer key

5.22.2 Process: GATE CSE 2001 | Question: 2.20

Which of the following does not interrupt a running process?

A. A device B. Timer C. Scheduler process D. Power failure

gatecse-2001 operating-system easy process

Answer key

5.22.3 Process: GATE CSE 2002 | Question: 2.21

Which combination of the following features will suffice to characterize an OS as a multi-programmed OS? 1

a. More than one program may be loaded into main memory at the same time for execution
b. If a program waits for certain events such as I/O, another program is immediately scheduled for execution
c. If the execution of a program terminates, another program is immediately scheduled for execution. 2

A. (a)
B. (a) and (b)
C. (a) and (c)
D. (a), (b) and (c) 3

gatecse-2002 operating-system normal process 4

Answer key

5.22.4 Process: GATE CSE 2023 | Question: 12

Which one or more of the following need to be saved on a context switch from one thread $(\mathrm{T1})$ of a process to another thread $(\mathrm{T2})$ of the same process? 5

A. Page table base register
B. Stack pointer
C. Program counter
D. General purpose registers 6

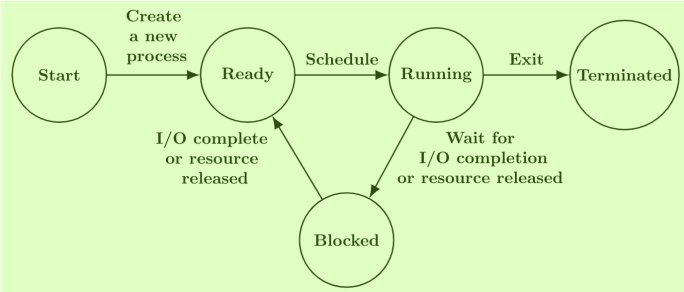
gatecse-2023 operating-system process multiple-selects one-mark 7

Answer key

5.22.5 Process: GATE IT 2006 | Question: 13

The process state transition diagram of an operating system is as given below. 8

Which of the following must be FALSE about the above operating system?



9

A. It is a multiprogrammed operating system
B. It uses preemptive scheduling
C. It uses non-preemptive scheduling
D. It is a multi-user operating system 10

gateit-2006 operating-system normal process 11

Answer key 12

5.23 Process Scheduling (49)

✍ **Practice Tests:** Test 1 (15Q) | Test 2 (15Q) | Test 3 (15Q) | Test 4 (15Q) | Test 5 (2Q)

5.23.1 Process Scheduling: GATE CSE 1988 | Question: 2xa

State any undesirable characteristic of the following criteria for measuring performance of an operating system:

Turn around time

gate1988 normal descriptive operating-system process-scheduling

Answer key

5.23.2 Process Scheduling: GATE CSE 1988 | Question: 2xb

State any undesirable characteristic of the following criteria for measuring performance of an operating system: 1

Waiting time 2

gate1988 normal descriptive operating-system process-scheduling 3

Answer key

5.23.3 Process Scheduling: GATE CSE 1990 | Question: 1-vi

The highest-response ratio next scheduling policy favours _____ jobs, but it also limits the waiting time of _____ jobs. 4

gate1990 operating-system process-scheduling fill-in-the-blanks 5

Answer key

5.23.4 Process Scheduling: GATE CSE 1993 | Question: 7.10

Assume that the following jobs are to be executed on a single processor system 6

Job Id	CPU Burst Time
p	4
q	1
r	8
s	1
t	2

7

The jobs are assumed to have arrived at time 0^+ and in the order p, q, r, s, t. Calculate the departure time (completion time) for job p if scheduling is round robin with time slice 1 8

A. 4 B. 10 C. 11 D. 12 9

E. None of the above 10

gate1993 operating-system process-scheduling normal 11

Answer key

5.23.5 Process Scheduling: GATE CSE 1995 | Question: 1.15

Which scheduling policy is most suitable for a time shared operating system? 12

A. Shortest Job First
B. Round Robin 13
C. First Come First Serve
D. Elevator

gate1995 operating-system process-scheduling easy 14

Answer key 15

5.23.6 Process Scheduling: GATE CSE 1995 | Question: 2.6

The sequence _____ is an optimal non-preemptive scheduling sequence for the following jobs which leaves the CPU idle for _____ unit(s) of time.

Job	Arrival Time	Burst Time
1	0.0	9
2	0.6	5
3	1.0	1

A. $\{3,2,1\},1$ B. $\{2,1,3\},0$ C. $\{3,2,1\},0$ D. $\{1,2,3\},5$

gate1995 operating-system process-scheduling normal

Answer key

5.23.7 Process Scheduling: GATE CSE 1996 | Question: 2.20, ISRO2008-15

Four jobs to be executed on a single processor system arrive at time 0 in the order A, B, C, D. Their burst CPU time requirements are $4, 1, 8, 1$ time units respectively. The completion time of A under round robin scheduling with time slice of one time unit is 1

A. 10 B. 4 C. 8 D. 9 2

gate1996 operating-system process-scheduling normal isro2008 3

Answer key

5.23.8 Process Scheduling: GATE CSE 1998 | Question: 2.17, UGCNET-Dec2012-III: 43 4

Consider n processes sharing the CPU in a round-robin fashion. Assuming that each process switch takes s seconds, what must be the quantum size q such that the overhead resulting from process switching is minimized but at the same time each process is guaranteed to get its turn at the CPU at least every t seconds? 5

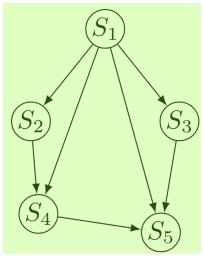
A. $q \leq \frac{t-ns}{n-1}$ B. $q \geq \frac{t-ns}{n-1}$ 6
C. $q \leq \frac{t-ns}{n+1}$ D. $q \geq \frac{t-ns}{n+1}$

gate1998 operating-system process-scheduling normal ugcnetcse-dec2012-paper3 7

Answer key 8

5.23.9 Process Scheduling: GATE CSE 1998 | Question: 24

a. Four jobs are waiting to be run. Their expected run times are $6, 3, 5$ and x. In what order should they be run to minimize the average response time? 9
b. Write a concurrent program using par begin-par end to represent the precedence graph shown below.



10

gate1998 operating-system process-scheduling descriptive

Answer key

5.23.10 Process Scheduling: GATE CSE 1998 | Question: 7-b

In a computer system where the 'best-fit' algorithm is used for allocating 'jobs' to 'memory partitions', the following situation was encountered: 11

Partitions size in KB	$4K\ 8K\ 20K\ 2K$
Job sizes in KB	$2K\ 14K\ 3K\ 6K\ 6K\ 10K\ 20K\ 2K$
Time for execution	$4\ 10\ 2\ 1\ 4\ 1\ 8\ 6$

12

When will the $20K$ job complete? 13

gate1998 operating-system process-scheduling normal 14

Answer key

5.23.11 Process Scheduling: GATE CSE 2002 | Question: 1.22

Which of the following scheduling algorithms is non-preemptive?

A. Round Robin
B. First-In First-Out
C. Multilevel Queue Scheduling
D. Multilevel Queue Scheduling with Feedback

1

gatecse-2002 operating-system process-scheduling easy

Answer key

5.23.12 Process Scheduling: GATE CSE 2003 | Question: 77

2

A uni-processor computer system only has two processes, both of which alternate 10 ms CPU bursts with 90 ms I/O bursts. Both the processes were created at nearly the same time. The I/O of both processes can proceed in parallel. Which of the following scheduling strategies will result in the *least* CPU utilization (over a long period of time) for this system?

A. First come first served scheduling
B. Shortest remaining time first scheduling
C. Static priority scheduling with different priorities for the two processes
D. Round robin scheduling with a time quantum of 5 ms

3

gatecse-2003 operating-system process-scheduling normal

Answer key

5.23.13 Process Scheduling: GATE CSE 2004 | Question: 46

4

Consider the following set of processes, with the arrival times and the CPU-burst times gives in milliseconds.

5

Process	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	3
P4	4	1

What is the average turnaround time for these processes with the preemptive shortest remaining processing time first (SRPT) algorithm?

6

A. 5.50
B. 5.75
C. 6.00
D. 6.25

7

gatecse-2004 operating-system process-scheduling normal

8

Answer key

5.23.14 Process Scheduling: GATE CSE 2006 | Question: 06, ISRO2009-14

9

Consider three CPU-intensive processes, which require 10, 20 and 30 time units and arrive at times 0, 2 and 6, respectively. How many context switches are needed if the operating system implements a shortest remaining time first scheduling algorithm? Do not count the context switches at time zero and at the end.

A. 1
B. 2
C. 3
D. 4

10

gatecse-2006 operating-system process-scheduling normal isro2009

Answer key

5.23.15 Process Scheduling: GATE CSE 2006 | Question: 64

Consider three processes (process id 0, 1, 2 respectively) with compute time bursts 2, 4 and 8 time units. All processes arrive at time zero. Consider the longest remaining time first (LRTF) scheduling algorithm. In LRTF ties are broken by giving priority to the process with the lowest process id. The average turn around time is:

A. 13 units
B. 14 units
C. 15 units
D. 16 units

gatecse-2006 operating-system process-scheduling normal

Answer key

5.23.16 Process Scheduling: GATE CSE 2006 | Question: 65

Consider three processes, all arriving at time zero, with total execution time of 10, 20 and 30 units, respectively. Each process spends the first 20% of execution time doing I/O, the next 70% of time doing computation, and the last 10% of time doing I/O again. The operating system uses a shortest remaining compute time first scheduling algorithm and schedules a new process either when the running process gets blocked on I/O or when the running process finishes its compute burst. Assume that all I/O operations can be overlapped as much as possible. For what percentage of time does the CPU remain idle?

A. 0% B. 10.6% C. 30.0% D. 89.4%

gatecse-2006 operating-system process-scheduling normal

Answer key

5.23.17 Process Scheduling: GATE CSE 2007 | Question: 16

Group 1 contains some CPU scheduling algorithms and Group 2 contains some applications. Match entries in Group 1 to entries in Group 2.

Group I		Group II	
(P)	Gang Scheduling	(1)	Guaranteed Scheduling
(Q)	Rate Monotonic Scheduling	(2)	Real-time Scheduling
(R)	Fair Share Scheduling	(3)	Thread Scheduling

A. $P-3; Q-2; R-1$ B. $P-1; Q-2; R-3$ C. $P-2; Q-3; R-1$ D. $P-1; Q-3; R-2$

gatecse-2007 operating-system process-scheduling normal

Answer key

5.23.18 Process Scheduling: GATE CSE 2007 | Question: 55

An operating system used Shortest Remaining System Time first (SRT) process scheduling algorithm. Consider the arrival times and execution times for the following processes:

Process	Execution Time	Arrival Time
P1	20	0
P2	25	15
P3	10	30
P4	15	45

What is the total waiting time for process $P2$?

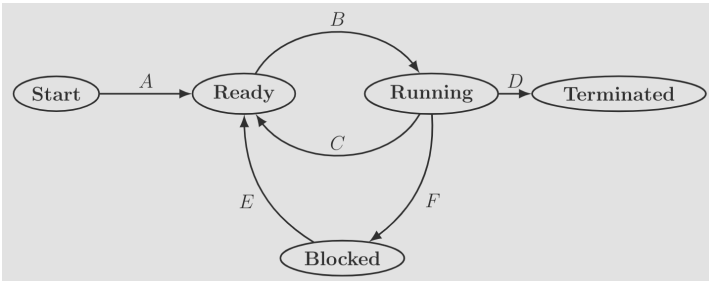
A. 5 B. 15 C. 40 D. 55

gatecse-2007 operating-system process-scheduling normal

Answer key

5.23.19 Process Scheduling: GATE CSE 2009 | Question: 32

In the following process state transition diagram for a uniprocessor system, assume that there are always some processes in the ready state:



Now consider the following statements: 1

I. If a process makes a transition D, it would result in another process making transition A immediately. 2
II. A process P_2 in blocked state can make transition E while another process P_1 is in running state.
III. The OS uses preemptive scheduling.
IV. The OS uses non-preemptive scheduling.

Which of the above statements are TRUE? 3

A. I and II B. I and III C. II and III D. II and IV 4

gatecse-2009 operating-system process-scheduling normal 5

Answer key 6

5.23.20 Process Scheduling: GATE CSE 2010 | Question: 25

Which of the following statements are true? 7

I. Shortest remaining time first scheduling may cause starvation 8
II. Preemptive scheduling may cause starvation
III. Round robin is better than FCFS in terms of response time

A. I only B. I and III only C. II and III only D. I, II and III 9

gatecse-2010 operating-system process-scheduling easy 10

Answer key

5.23.21 Process Scheduling: GATE CSE 2011 | Question: 35

Consider the following table of arrival time and burst time for three processes $P0, P1$ and $P2$. 11

Process	Arrival Time	Burst Time
P0	0 ms	9
P1	1 ms	4
P2	2 ms	9

12

The pre-emptive shortest job first scheduling algorithm is used. Scheduling is carried out only at arrival or completion of processes. What is the average waiting time for the three processes? 13

A. 5.0 ms B. 4.33 ms C. 6.33 ms D. 7.33 ms 14

gatecse-2011 operating-system process-scheduling normal 15

Answer key 16

5.23.22 Process Scheduling: GATE CSE 2012 | Question: 31

Consider the 3 processes, $P1, P2$ and $P3$ shown in the table. 17

Process	Arrival Time	Time Units Required
P1	0	5
P2	1	7
P3	3	4

18

The completion order of the 3 processes under the policies FCFS and RR2 (round robin scheduling with CPU quantum of 2 time units) are 19

A. **FCFS**: $P1, P2, P3$ **RR2**: $P1, P2, P3$ 20
B. **FCFS**: $P1, P3, P2$ **RR2**: $P1, P3, P2$
C. **FCFS**: $P1, P2, P3$ **RR2**: $P1, P3, P2$
D. **FCFS**: $P1, P3, P2$ **RR2**: $P1, P2, P3$

gatecse-2012 operating-system process-scheduling normal

Answer key

5.23.23 Process Scheduling: GATE CSE 2013 | Question: 10

A scheduling algorithm assigns priority proportional to the waiting time of a process. Every process starts with zero (the lowest priority). The scheduler re-evaluates the process priorities every T time units and decides the next process to schedule. Which one of the following is **TRUE** if the processes have no I/O operations and all arrive at time zero?

A. This algorithm is equivalent to the first-come-first-serve algorithm.
B. This algorithm is equivalent to the round-robin algorithm.
C. This algorithm is equivalent to the shortest-job-first algorithm.
D. This algorithm is equivalent to the shortest-remaining-time-first algorithm.

gatecse-2013 operating-system process-scheduling normal

Answer key

5.23.24 Process Scheduling: GATE CSE 2014 | Set 1 | Question: 32

Consider the following set of processes that need to be scheduled on a single CPU. All the times are given in milliseconds.

Process Name	Arrival Time	Execution Time
A	0	6
B	3	2
C	5	4
D	7	6
E	10	3

Using the *shortest remaining time first* scheduling algorithm, the average process turnaround time (in msec) is _____.

gatecse-2014-set1 operating-system process-scheduling numerical-answers normal

Answer key

5.23.25 Process Scheduling: GATE CSE 2014 | Set 2 | Question: 32

Three processes A, B and C each execute a loop of 100 iterations. In each iteration of the loop, a process performs a single computation that requires t_c CPU milliseconds and then initiates a single I/O operation that lasts for t_{io} milliseconds. It is assumed that the computer where the processes execute has sufficient number of I/O devices and the OS of the computer assigns different I/O devices to each process. Also, the scheduling overhead of the OS is negligible. The processes have the following characteristics:

Process id	t_c	t_{io}
A	100 ms	500 ms
B	350 ms	500 ms
C	200 ms	500 ms

The processes A, B, and C are started at times 0, 5 and 10 milliseconds respectively, in a pure time sharing system (round robin scheduling) that uses a time slice of 50 milliseconds. The time in milliseconds at which process C would ***complete*** its first I/O operation is _____.

gatecse-2014-set2 operating-system process-scheduling numerical-answers normal

Answer key

5.23.26 Process Scheduling: GATE CSE 2014 | Set 3 | Question: 32

An operating system uses *shortest remaining time first* scheduling algorithm for pre-emptive scheduling of processes. Consider the following set of processes with their arrival times and CPU burst times (in

milliseconds): 1

Process	Arrival Time	Burst Time
P1	0	12
P2	2	4
P3	3	6
P4	8	5

2

The average waiting time (in milliseconds) of the processes is _____. 3

gatecse-2014-set3 operating-system process-scheduling numerical-answers normal 4

Answer key

5.23.27 Process Scheduling: GATE CSE 2015 | Set 1 | Question: 46

5

Consider a uniprocessor system executing three tasks T_1, T_2 and T_3 each of which is composed of an infinite sequence of jobs (or instances) which arrive periodically at intervals of $3, 7$ and 20 milliseconds, respectively. The priority of each task is the inverse of its period, and the available tasks are scheduled in order of priority, which is the highest priority task scheduled first. Each instance of T_1, T_2 and T_3 requires an execution time of $1, 2$ and 4 milliseconds, respectively. Given that all tasks initially arrive at the beginning of the 1^{st} millisecond and task preemptions are allowed, the first instance of T_3 completes its execution at the end of_____milliseconds.

gatecse-2015-set1 operating-system process-scheduling normal numerical-answers 6

Answer key

5.23.28 Process Scheduling: GATE CSE 2015 | Set 3 | Question: 1

The maximum number of processes that can be in $Ready$ state for a computer system with n CPUs is : 7

A. n B. n^2 C. 2^n D. Independent of n 8

gatecse-2015-set3 operating-system process-scheduling easy 9

Answer key

5.23.29 Process Scheduling: GATE CSE 2015 | Set 3 | Question: 34

10

For the processes listed in the following table, which of the following scheduling schemes will give the lowest average turnaround time?

Process	Arrival Time	Process Time
A	0	3
B	1	6
C	4	4
D	6	2

11

A. First Come First Serve
B. Non-preemptive Shortest job first
C. Shortest Remaining Time
D. Round Robin with Quantum value two

12

gatecse-2015-set3 operating-system process-scheduling normal 13

Answer key

5.23.30 Process Scheduling: GATE CSE 2016 | Set 1 | Question: 20

Consider an arbitrary set of CPU-bound processes with unequal CPU burst lengths submitted at the same time to a computer system. Which one of the following process scheduling algorithms would minimize the average waiting time in the ready queue?

A. Shortest remaining time first
B. Round-robin with the time quantum less than the shortest CPU burst
C. Uniform random

D. Highest priority first with priority proportional to CPU burst length

gatecse-2016-set1 operating-system process-scheduling normal

Answer key

5.23.31 Process Scheduling: GATE CSE 2016 | Set 2 | Question: 47

Consider the following processes, with the arrival time and the length of the CPU burst given in milliseconds. 1
The scheduling algorithm used is preemptive shortest remaining-time first.

Process	Arrival Time	Burst Time
P_1	0	10
P_2	3	6
P_3	7	1
P_4	8	3

2

The average turn around time of these processes is _____ milliseconds. 3

gatecse-2016-set2 operating-system process-scheduling normal numerical-answers 4

Answer key

5.23.32 Process Scheduling: GATE CSE 2017 | Set 1 | Question: 24

Consider the following CPU processes with arrival times (in milliseconds) and length of CPU bursts (in milliseconds) as given below: 5

Process	Arrival Time	Burst Time
P_1	0	7
P_2	3	3
P_3	5	5
P_4	6	2

6

If the pre-emptive shortest remaining time first scheduling algorithm is used to schedule the processes, then the average waiting time across all processes is _____ milliseconds. 7

gatecse-2017-set1 operating-system process-scheduling numerical-answers 8

Answer key

5.23.33 Process Scheduling: GATE CSE 2017 | Set 2 | Question: 51

Consider the set of process with arrival time (in milliseonds), CPU burst time (in millisecods) and priority (0 is the highest priority) shown below. None of the process have I/O burst time 9

Process	Arrival Time	Burst Time	Priority
P_1	0	11	2
P_2	5	28	0
P_3	12	2	3
P_4	2	10	1
P_5	9	16	4

10

The average waiting time (in milli seconds) of all the process using premtive priority scheduling algorithm is _____ 11

gatecse-2017-set2 operating-system process-scheduling numerical-answers 12

Answer key

5.23.34 Process Scheduling: GATE CSE 2019 | Question: 41

Consider the following four processes with arrival times (in milliseconds) and their length of CPU bursts (in milliseconds) as shown below:

Process	$P1$	$P2$	$P3$	$P4$
Arrival Time	0	1	3	4
CPU burst time	3	1	3	Z

1

These processes are run on a single processor using preemptive Shortest Remaining Time First scheduling algorithm. If the average waiting time of the processes is 1 millisecond, then the value of Z is _____ 2

gatecse-2019 numerical-answers operating-system process-scheduling two-marks 3

Answer key

5.23.35 Process Scheduling: GATE CSE 2020 | Question: 12

Consider the following statements about process state transitions for a system using preemptive scheduling. 4

I. A running process can move to ready state. 5
II. A ready process can move to running state.
III. A blocked process can move to running state.
IV. A blocked process can move to ready state.

Which of the above statements are TRUE? 6

A. I, II, and III only
B. II and III only
C. I, II, and IV only
D. I, II, III and IV only 7

gatecse-2020 operating-system process-scheduling one-mark easy

Answer key

5.23.36 Process Scheduling: GATE CSE 2020 | Question: 50

Consider the following set of processes, assumed to have arrived at time 0. Consider the CPU scheduling algorithms Shortest Job First (SJF) and Round Robin (RR). For RR, assume that the processes are scheduled in the order P_1, P_2, P_3, P_4. 8

Processes	P_1	P_2	P_3	P_4
Burst time (in ms)	8	7	2	4

9

If the time quantum for RR is 4 ms, then the absolute value of the difference between the average turnaround times (in ms) of SJF and RR (round off to 2 decimal places is_____ 10

gatecse-2020 numerical-answers operating-system process-scheduling two-marks 11

Answer key

5.23.37 Process Scheduling: GATE CSE 2021 | Set 1 | Question: 25

Three processes arrive at time zero with CPU bursts of 16, 20 and 10 milliseconds. If the scheduler has prior knowledge about the length of the CPU bursts, the minimum achievable average waiting time for these three processes in a non-preemptive scheduler (rounded to nearest integer) is _____ milliseconds. 12

gatecse-2021-set1 operating-system process-scheduling numerical-answers one-mark 13

Answer key

5.23.38 Process Scheduling: GATE CSE 2021 | Set 2 | Question: 14

Which of the following statement(s) is/are correct in the context of CPU scheduling?

A. Turnaround time includes waiting time
B. The goal is to only maximize CPU utilization and minimize throughput
C. Round-robin policy can be used even when the CPU time required by each of the processes is not known apriori
D. Implementing preemptive scheduling needs hardware support

gatecse-2021-set2 multiple-selects operating-system process-scheduling one-mark

Answer key

5.23.39 Process Scheduling: GATE CSE 2023 | Question: 17

Which one or more of the following CPU scheduling algorithms can potentially cause starvation? 1

A. First-in First-Out
B. Round Robin 2
C. Priority Scheduling
D. Shortest Job First

gatecse-2023 operating-system process-scheduling multiple-selects one-mark 3

Answer key

5.23.40 Process Scheduling: GATE CSE 2024 | Set 1 | Question: 15

4

Which of the following process state transitions is/are NOT possible? 5

A. Running to Ready
B. Waiting to Running 6
C. Ready to Waiting
D. Running to Terminated

gatecse-2024-set1 operating-system process-scheduling multiple-selects one-mark 7

Answer key

5.23.41 Process Scheduling: GATE CSE 2024 | Set 2 | Question: 27

Consider a single processor system with four processes A, B, C, and D, represented as given below, 8 where for each process the first value is its arrival time, and the second value is its CPU burst time.

$$\mathrm{A}(0,10), \mathrm{B}(2,6), \mathrm{C}(4,3), \text{ and } \mathrm{D}(6,7).$$

Which one of the following options gives the average waiting times when preemptive Shortest Remaining Time First (SRTF) and Non-Preemptive Shortest Job First (NP-SJF) CPU scheduling algorithms are applied to the processes? 9

A. $\mathrm{SRTF} = 6, \mathrm{NP-SJF} = 7$ 10
B. $\mathrm{SRTF} = 6, \mathrm{NP-SJF} = 7.5$
C. $\mathrm{SRTF} = 7, \mathrm{NP-SJF} = 7.5$
D. $\mathrm{SRTF} = 7, \mathrm{NP-SJF} = 8.5$

gatecse-2024-set2 operating-system process-scheduling two-marks

Answer key

5.23.42 Process Scheduling: GATE CSE 2025 | Set 1 | Question: 28

A computer has two processors, M_1 and M_2. Four processes P_1, P_2, P_3, P_4 with CPU bursts of $20, 16, 25$, and 10 milliseconds, respectively, arrive at the same time and these are the only processes in the system. The scheduler uses non-preemptive priority scheduling, with priorities decided as follows: 11

- M_1 uses priority of execution for the processes as, $P_1 > P_3 > P_2 > P_4$, i.e., P_1 and P_4 have highest and lowest priorities, respectively. 12
- M_2 uses priority of execution for the processes as, $P_2 > P_3 > P_4 > P_1$, i.e., P_2 and P_1 have highest and lowest priorities, respectively.

A process P_i is scheduled to a processor M_k, if the processor is free and no other process P_j is waiting with higher priority. At any given point of time, a process can be allocated to any one of the free processors without violating the execution priority rules. Ignore the context switch time. What will be the average waiting time of the processes in milliseconds? 13

A. 9.00 B. 8.75 C. 6.50 D. 7.50 14

gatecse2025-set1 operating-system process-scheduling two-marks

Answer key

5.23.43 Process Scheduling: GATE CSE 2026 | Set 1 | Question: 54

Consider a CPU that has to execute two types of processes. The first type, Actuators (A), requires a CPU burst of 6 seconds. The second type, Controllers (C), requires a CPU burst of 8 seconds. A new process of type A arrives at time $t = 10, 20, 30, 40,$ and 50 (in seconds). Similarly, a new process of type C arrives at time $t = 11, 22, 33, 44,$ and 55 (in seconds). The CPU scheduling policy is First Come First Serve (FCFS). The first process of type A starts running at $t = 10$ seconds. The average waiting time (in seconds) for the 10 processes is _____. (rounded off to one decimal place)

gatecse-2026-set1 numerical-answers operating-system process-scheduling two-marks

Answer key

5.23.44 Process Scheduling: GATE CSE 2026 | Set 2 | Question: 13

Which one of the following CPU scheduling algorithms **cannot** be preemptive?

A. Shortest Remaining Time First (SRTF) Scheduling
B. First Come First Serve (FCFS) Scheduling
C. Round Robin Scheduling
D. Priority Scheduling

gatecse-2026-set2 operating-system process-scheduling one-mark

Answer key

5.23.45 Process Scheduling: GATE IT 2005 | Question: 60

We wish to schedule three processes $P1$, $P2$ and $P3$ on a uniprocessor system. The priorities, CPU time requirements and arrival times of the processes are as shown below.

Process	Priority	CPU time required	Arrival time (hh:mm:ss)
P1	10 (highest)	20 sec	00 : 00 : 05
P2	9	10 sec	00 : 00 : 03
P3	8 (lowest)	15 sec	00 : 00 : 00

We have a choice of preemptive or non-preemptive scheduling. In preemptive scheduling, a late-arriving higher priority process can preempt a currently running process with lower priority. In non-preemptive scheduling, a late-arriving higher priority process must wait for the currently executing process to complete before it can be scheduled on the processor.

What are the turnaround times (time from arrival till completion) of $P2$ using preemptive and non-preemptive scheduling respectively?

A. 30 sec, 30 sec
B. 30 sec, 10 sec
C. 42 sec, 42 sec
D. 30 sec, 42 sec

gateit-2005 operating-system process-scheduling normal

Answer key

5.23.46 Process Scheduling: GATE IT 2006 | Question: 12

In the working-set strategy, which of the following is done by the operating system to prevent thrashing?

I. It initiates another process if there are enough extra frames.
II. It selects a process to suspend if the sum of the sizes of the working-sets exceeds the total number of available frames.

A. I only
B. II only
C. Neither I nor II
D. Both I and II

gateit-2006 operating-system process-scheduling normal

Answer key

5.23.47 Process Scheduling: GATE IT 2006 | Question: 54

The arrival time, priority, and duration of the CPU and I/O bursts for each of three processes P_1, P_2 and P_3 are given in the table below. Each process has a CPU burst followed by an I/O burst followed by another

CPU burst. Assume that each process has its own I/O resource.

Process	Arrival Time	Priority	Burst duration (CPU)	Burst duration (I/O)	Burst duration) (CPU)
P_1	0	2	1	5	3
P_2	2	3 (lowest)	3	3	1
P_3	3	1 (highest)	2	3	1

The multi-programmed operating system uses preemptive priority scheduling. What are the finish times of the processes P_1, P_2 and P_3?

A. $11, 15, 9$ B. $10, 15, 9$ C. $11, 16, 10$ D. $12, 17, 11$

gateit-2006 operating-system process-scheduling normal

Answer key

5.23.48 Process Scheduling: GATE IT 2007 | Question: 26

Consider n jobs $J_1, J_2 \ldots J_n$ such that job J_i has execution time t_i and a non-negative integer weight w_i. The weighted mean completion time of the jobs is defined to be $\frac{\sum_{i=1}^{n} w_i T_i}{\sum_{i=1}^{n} w_i}$, where T_i is the completion time of job J_i. Assuming that there is only one processor available, in what order must the jobs be executed in order to minimize the weighted mean completion time of the jobs?

A. Non-decreasing order of t_i
B. Non-increasing order of w_i
C. Non-increasing order of $w_i t_i$
D. Non-increasing order of w_i / t_i

gateit-2007 operating-system process-scheduling normal

Answer key

5.23.49 Process Scheduling: GATE IT 2008 | Question: 55

If the time-slice used in the round-robin scheduling policy is more than the maximum time required to execute any process, then the policy will

A. degenerate to shortest job first
B. degenerate to priority scheduling
C. degenerate to first come first serve
D. none of the above

gateit-2008 operating-system process-scheduling easy

Answer key

5.24 Process Synchronization (52)

Practice Tests: Test 1 (15Q) Test 2 (15Q) Test 3 (11Q) Weekly Quiz 1 (16Q)

5.24.1 Process Synchronization: GATE CSE 1987 | Question: 1-xvi

A critical region is

A. One which is enclosed by a pair of P and V operations on semaphores.
B. A program segment that has not been proved bug-free.
C. A program segment that often causes unexpected system crashes.
D. A program segment where shared resources are accessed.

gate1987 operating-system process-synchronization

Answer key

5.24.2 Process Synchronization: GATE CSE 1987 | Question: 8a

Consider the following proposal to the "readers and writers problem."
Shared variables and semaphores:

```
aw, ar, rw, rr : interger;
mutex, reading, writing: semaphore:
initial values of variables and states of semaphores:
```

```
ar=rr=aw=rw=0
reading_value = writing_value = 0
mutex_value = 1.                    Process writer;
Process reader;                       begin
begin                               while true do
repeat                              begin
       P(mutex);                           P(mutex);
       ar := ar+1;                        aw := aw + 1;
       grantread;                          grantwrite;
       V(mutex);                           V(mutex);
       P(reading);                         P(writing);
       read;                        Write;
       P(mutex);                           P(mutex);
       rr := rr - 1;                rw := rw - 1;
       ar := ar - 1;                ar := aw - 1;
       grantwrite;                  grantread;
       V(mutex);                    V(mutex);
       other-work;                  other-work;
until false                    end
end.                          end.

Procedure grantread;
begin
   if aw = 0
   then while (rr < ar) do
      begin rr := rr + 1;
         V (reading)
      end
end;
Procedure grantwrite;
begin
   if rr = 0
   then while (rw < aw) do
      begin rw := rw + 1;
         V (writing)
      end
end;
```

1

a. Give the value of the shared variables and the states of semaphores when 12 readers are reading and writers are writing. 2
b. Can a group of readers make waiting writers starve? Can writers starve readers?
c. Explain in two sentences why the solution is incorrect.

gate1987 operating-system process-synchronization descriptive

Answer key

5.24.3 Process Synchronization: GATE CSE 1988 | Question: 10iib

Given below is solution for the critical section problem of two processes P_0 and P_1 sharing the following 3 variables:

```
    var flag :array [0..1] of boolean; (initially false)
      turn: 0 .. 1;
```

4

The program below is for process $P_i (i = 0 \text{ or } 1)$ where process $P_j (j = 1 \text{ or } 0)$ being the other one. 5

```
repeat
      flag[i]:= true;
      while turn != i
      do begin
         while flag [j] do skip
         turn:=i;
      end

      critical section

      flag[i]:=false;
until false
```

6

Determine of the above solution is correct. If it is incorrect, demonstrate with an example how it violates the conditions. 7

gate1988 descriptive operating-system process-synchronization

Answer key

5.24.4 Process Synchronization: GATE CSE 1990 | Question: 2-iii

Match the pairs: 1

(a)	Critical region	(p)	Hoare's monitor
(b)	Wait/Signal	(q)	Mutual exclusion
(c)	Working Set	(r)	Principle of locality
(d)	Deadlock	(s)	Circular Wait

2

match-the-following gate1990 operating-system process-synchronization 3

Answer key

5.24.5 Process Synchronization: GATE CSE 1991 | Question: 11,a

Consider the following scheme for implementing a critical section in a situation with three processes P_i, P_j and P_k. 4

```
Pi;
repeat
   flag[i] := true;
   while flag [j] or flag[k] do
      case turn of
      j: if flag [j] then
      begin
         flag [i] := false;
         while turn != i do skip;
         flag [i] := true;
      end;
      k: if flag [k] then
      begin
         flag [i] := false,
         while turn != i do skip;
         flag [i] := true
      end
   end
   critical section
   if turn = i then turn := j;
      flag [i] := false
   non-critical section
until false;
```

5

a. Does the scheme ensure mutual exclusion in the critical section? Briefly explain. 6

gate1991 process-synchronization normal operating-system descriptive

Answer key

5.24.6 Process Synchronization: GATE CSE 1991 | Question: 11,b

Consider the following scheme for implementing a critical section in a situation with three processes P_i, P_j and P_k. 7

Pi; 8

```
repeat
   flag[i] := true;
   while flag [j] or flag[k] do
      case turn of
      j: if flag [j] then
      begin
         flag [i] := false;
         while turn != i do skip;
         flag [i] := true;
      end;
      k: if flag [k] then
      begin
         flag [i] := false,
         while turn != i do skip;
         flag [i] := true
      end
   end
```

9

```
   critical section
   if turn = i then turn := j;
      flag [i] := false
   non-critical section
until false;
```

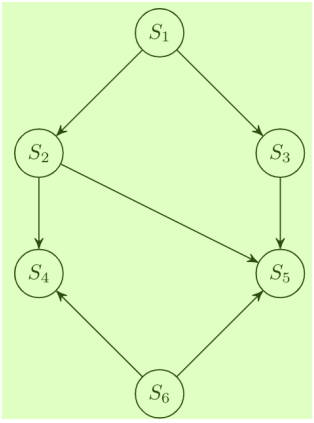
Is there a situation in which a waiting process can never enter the critical section? If so, explain and suggest modifications to the code to solve this problem

gate1991 process-synchronization normal operating-system descriptive

Answer key

5.24.7 Process Synchronization: GATE CSE 1993 | Question: 22

Write a concurrent program using parbegin-parend and semaphores to represent the precedence constraints of the statements S_1 to S_6, as shown in figure below.



gate1993 operating-system process-synchronization normal descriptive

Answer key

5.24.8 Process Synchronization: GATE CSE 1994 | Question: 27

A. Draw a precedence graph for the following sequential code. The statements are numbered from S_1 to S_6

S_1 read n
S_2 i := 1
S_3 if i > n next
S_4 a(i) := i+1
S_5 i := i+1
S_6 next : write a(i)

B. Can this graph be converted to a concurrent program using parbegin-parend construct only?

gate1994 operating-system process-synchronization normal descriptive

Answer key

5.24.9 Process Synchronization: GATE CSE 1995 | Question: 19

Consider the following program segment for concurrent processing using semaphore operators P and V for synchronization. Draw the precedence graph for the statements S_1 to S_9.

```
var
a,b,c,d,e,f,g,h,i,j,k : semaphore;
begin
cobegin
   begin S1; V(a); V(b) end;
   begin P(a); S2; V(c); V(d) end;
   begin P(c); S4; V(e) end;
   begin P(d); S5; V(f) end;
   begin P(e); P(f); S7; V(k) end
```

```
    begin P(b); S3; V(g); V(h) end;
    begin P(g); S6; V(i) end;
    begin P(h); P(i); S8; V(j) end;
    begin P(j); P(k); S9 end;
coend
end;
```

1

gate1995 operating-system process-synchronization normal descriptive 2

Answer key

5.24.10 Process Synchronization: GATE CSE 1996 | Question: 1.19, ISRO2008-61

3

A critical section is a program segment 4

A. which should run in a certain amount of time
B. which avoids deadlocks
C. where shared resources are accessed
D. which must be enclosed by a pair of semaphore operations, P and V

5

gate1996 operating-system process-synchronization easy isro2008 6

Answer key 7

5.24.11 Process Synchronization: GATE CSE 1996 | Question: 2.19

8

A solution to the Dining Philosophers Problem which avoids deadlock is to 9

A. ensure that all philosophers pick up the left fork before the right fork
B. ensure that all philosophers pick up the right fork before the left fork
C. ensure that one particular philosopher picks up the left fork before the right fork, and that all other philosophers pick up the right fork before the left fork
D. None of the above

10

gate1996 operating-system process-synchronization normal

Answer key

5.24.12 Process Synchronization: GATE CSE 1996 | Question: 21

The concurrent programming constructs fork and join are as below: 11

Fork <label> which creates a new process executing from the specified label 12

Join <variable> which decrements the specified synchronization variable (by 1) and terminates the process if the new value is not 0. 13

Show the precedence graph for $S1$, $S2$, $S3$, $S4$, and $S5$ of the concurrent program below. 14

```
N=2
M=2
Fork L3
Fork L4
S1
L1 : join N
S3
L2 : join M
S5
L3:S2
Goto L1
L4:S4
Goto L2
Next:
```

15

gate1996 operating-system process-synchronization normal descriptive

Answer key

5.24.13 Process Synchronization: GATE CSE 1997 | Question: 6.8

Each Process $P_i, i = 1 \ldots 9$ is coded as follows 1

```
repeat
    P(mutex)
    {Critical section}
    V(mutex)
forever
```
2

The code for P_{10} is identical except it uses V(mutex) in place of P(mutex). What is the largest number of processes that can be inside the critical section at any moment? 3

A. 1 B. 2 C. 3 D. None 4

gate1997 operating-system process-synchronization normal 5

Answer key

5.24.14 Process Synchronization: GATE CSE 1997 | Question: 73

A concurrent system consists of 3 processes using a shared resource R in a non-preemptible and mutually exclusive manner. The processes have unique priorities in the range $1 \ldots 3$, 3 being the highest priority. It is required to synchronize the processes such that the resource is always allocated to the highest priority requester. The pseudo code for the system is as follows. 6

Shared data 7

```
mutex:semaphore = 1:/* initialized to 1*/

process[3]:semaphore = 0; /*all initialized to 0 */

R_requested [3]:boolean = false; /*all initialized to flase */

busy: boolean = false; /*initialized to false */
```
8

Code for processes 9

```
begin process
my-priority:integer;
my-priority:=___; /*in the range 1..3*/
repeat
    request_R(my-priority);
    P (proceed [my-priority]);
    {use shared resource R}
    release_R (my-priority);
forever
end process;
```
10

Procedures 11

```
procedure request_R(priority);
P(mutex);
if busy = true then
    R_requested [priority]:=true;
else
begin
    V(proceed [priority]);
    busy:=true;
end
V(mutex)
```
12

Give the pseudo code for the procedure release_R. 13

gate1997 operating-system process-synchronization descriptive 14

Answer key

5.24.15 Process Synchronization: GATE CSE 1998 | Question: 1.30

When the result of a computation depends on the speed of the processes involved, there is said to be

A. cycle stealing B. race condition C. a time lock D. a deadlock

gate1998 operating-system easy process-synchronization

Answer key

5.24.16 Process Synchronization: GATE CSE 1999 | Question: 20-a

A certain processor provides a 'test and set' instruction that is used as follows: 1

```
TSET register, flag
```
2

This instruction atomically copies flag to register and sets flag to 1. Give pseudo-code for implementing the entry and exit code to a critical region using this instruction. 3

gate1999 operating-system process-synchronization normal descriptive 4

Answer key

5.24.17 Process Synchronization: GATE CSE 1999 | Question: 20-b

Consider the following solution to the producer-consumer problem using a buffer of size 1. Assume that the initial value of count is 0. Also assume that the testing of count and assignment to count are atomic operations. 5

```
Producer:
Repeat
   Produce an item;
   if count = 1 then sleep;
   place item in buffer.
   count = 1;
   Wakeup(Consumer);
Forever
```
6

```
Consumer:
Repeat
   if count = 0 then sleep;
   Remove item from buffer;
   count = 0;
   Wakeup(Producer);
   Consume item;
Forever;
```
7

Show that in this solution it is possible that both the processes are sleeping at the same time. 8

gate1999 operating-system process-synchronization normal descriptive

Answer key

5.24.18 Process Synchronization: GATE CSE 2000 | Question: 1.21

Let $m[0] \ldots m[4]$ be mutexes (binary semaphores) and $P[0] \ldots P[4]$ be processes. 9
Suppose each process $P[i]$ executes the following:

```
wait (m[i]); wait (m(i+1) mod 4]);
.....
release (m[i]); release (m(i+1) mod 4]);
```
10

This could cause 11

A. Thrashing
B. Deadlock
C. Starvation, but not deadlock
D. None of the above

12

gatecse-2000 operating-system process-synchronization normal 13

Answer key

5.24.19 Process Synchronization: GATE CSE 2000 | Question: 20

a. Fill in the boxes below to get a solution for the reader-writer problem, using a single binary semaphore, mutex (initialized to 1) and busy waiting. Write the box numbers (1, 2 and 3), and their contents in your answer book.

```
int R = 0, W = 0;

Reader () {
 L1:   wait (mutex);
    if (W == 0) {
       R = R + 1;
       ▭ _____(1)
    }
    else {
      ▭ _____(2)
       goto L1;
    }
    ..../* do the read*/
    wait (mutex);
    R = R - 1;
    signal (mutex);
}
```

1

```
Writer () {
 L2:   wait (mutex);
    if (▭) { _____ (3)
       signal (mutex);
       goto L2;
    }
    W=1;
    signal (mutex);
    ...../*do the write*/
    wait( mutex);
    W=0;
    signal (mutex);
}
```

2 3

b. Can the above solution lead to starvation of writers? 4

gatecse-2000 operating-system process-synchronization normal descriptive

Answer key

5.24.20 Process Synchronization: GATE CSE 2001 | Question: 2.22

Consider Peterson's algorithm for mutual exclusion between two concurrent processes i and j. The program executed by process is shown below. 5

```
repeat
   flag[i] = true;
   turn = j;
   while (P) do no-op;
   Enter critical section, perform actions, then
   exit critical section
   Flag[i] = false;
   Perform other non-critical section actions.
Until false;
```

6

For the program to guarantee mutual exclusion, the predicate P in the while loop should be 7

A. flag[j] = true and turn = i
B. flag[j] = true and turn = j 8
C. flag[i] = true and turn = j
D. flag[i] = true and turn = i

gatecse-2001 operating-system process-synchronization normal 9

Answer key

5.24.21 Process Synchronization: GATE CSE 2002 | Question: 18-a

Draw the process state transition diagram of an OS in which (i) each process is in one of the five states: created, ready, running, blocked (i.e., sleep or wait), or terminated, and (ii) only non-preemptive scheduling is used by the OS. Label the transitions appropriately.

gatecse-2002 operating-system process-synchronization normal descriptive

Answer key

5.24.22 Process Synchronization: GATE CSE 2002 | Question: 18-b

The functionality of atomic TEST-AND-SET assembly language instruction is given by the following function

```
int TEST-AND-SET (int *x)
{
   int y;
   A1: y=*x;
   A2: *x=1;
   A3: return y;
}
```

i. Complete the following C functions for implementing code for entering and leaving critical sections on the above TEST-AND-SET instruction.

```
int mutex=0;
void enter-cs()
{
   while(.....);

}
void leave-cs()
{ .....;

}
```

ii. Is the above solution to the critical section problem deadlock free and starvation-free?

iii. For the above solution, show by an example that mutual exclusion is not ensured if TEST-AND-SET instruction is not atomic?

gatecse-2002 operating-system process-synchronization normal descriptive

Answer key

5.24.23 Process Synchronization: GATE CSE 2002 | Question: 20

The following solution to the single producer single consumer problem uses semaphores for synchronization.

```
#define BUFFSIZE 100
buffer buf[BUFFSIZE];
int first = last = 0;
semaphore b_full = 0;
semaphore b_empty = BUFFSIZE

void producer()
{
while(1) {
   produce an item;
   p1:.....;
   put the item into buff (first);
   first = (first+1)%BUFFSIZE;
   p2: .....;
   }
}

void consumer()
{
while(1) {
   c1:.....
   take the item from buf[last];
   last = (last+1)%BUFFSIZE;
   c2:.....;
   consume the item;
   }
}
```

A. Complete the dotted part of the above solution.

B. Using another semaphore variable, insert one line statement each immediately after $p1$, immediately before $p2$, immediately after $c1$ and immediately before $c2$ so that the program works correctly for multiple producers and consumers.

gatecse-2002 operating-system process-synchronization normal descriptive

Answer key

5.24.24 Process Synchronization: GATE CSE 2003 | Question: 80

Suppose we want to synchronize two concurrent processes P and Q using binary semaphores S and T. 1
The code for the processes P and Q is shown below.

Process P:	Process Q:

while(1){
W:
print '0';
print '0';
X:
}
	while(1){
	Y:
	print '1';
	print '1';
	Z:
	}

2

Synchronization statements can be inserted only at points W, X, Y, and Z
Which of the following will always lead to an output staring with '001100110011'? 3

A. $P(S)$ at $W, V(S)$ at $X, P(T)$ at $Y, V(T)$ at Z, S and T initially 1
B. $P(S)$ at $W, V(T)$ at $X, P(T)$ at $Y, V(S)$ at Z, S initially 1, and T initially 0
C. $P(S)$ at $W, V(T)$ at $X, P(T)$ at $Y, V(S)$ at Z, S and T initially 1
D. $P(S)$ at $W, V(S)$ at $X, P(T)$ at $Y, V(T)$ at Z, S initially 1 , and T initially 0 4

gatecse-2003 operating-system process-synchronization normal

Answer key

5.24.25 Process Synchronization: GATE CSE 2003 | Question: 81

Suppose we want to synchronize two concurrent processes P and Q using binary semaphores S and T. 5
The code for the processes P and Q is shown below.

Process P:	Process Q:

while(1) {
W:
print '0';
print '0';
X:
}
	while(1) {
	Y:
	print '1';
	print '1';
	Z:
	}

6

Synchronization statements can be inserted only at points W, X, Y, and Z 7
Which of the following will ensure that the output string never contains a substring of the form 01^n0 and 10^n1 where n is odd?

A. $P(S)$ at $W, V(S)$ at $X, P(T)$ at $Y, V(T)$ at Z, S and T initially 1
B. $P(S)$ at $W, V(T)$ at $X, P(T)$ at $Y, V(S)$ at Z, S and T initially 1
C. $P(S)$ at $W, V(S)$ at $X, P(S)$ at $Y, V(S)$ at Z, S initially 1
D. $V(S)$ at $W, V(T)$ at $X, P(S)$ at $Y, P(T)$ at Z, S and T initially 1 8

gatecse-2003 operating-system process-synchronization normal

Answer key

5.24.26 Process Synchronization: GATE CSE 2004 | Question: 48

Consider two processes P_1 and P_2 accessing the shared variables X and Y protected by two binary

semaphores S_X and S_Y respectively, both initialized to 1. P and V denote the usual semaphore operators, where P decrements the semaphore value, and V increments the semaphore value. The pseudo-code of P_1 and P_2 is as follows: 1

P_1:	P_2:
While true do {	While true do {
L_1 :	L_3 : . . . . .
L_2 :	L_4 : . . . . .
X = X + 1;	Y = Y + 1;
$Y = Y - 1$;	$X = Y - 1$;
$V(S_X)$;	$V(S_Y)$;
$V(S_Y)$;	$V(S_X)$;
}	}

2

In order to avoid deadlock, the correct operators at L_1, L_2, L_3 and L_4 are respectively. 3

A. $P(S_Y), P(S_X); P(S_X), P(S_Y)$
B. $P(S_X), P(S_Y); P(S_Y), P(S_X)$ 4
C. $P(S_X), P(S_X); P(S_Y), P(S_Y)$
D. $P(S_X), P(S_Y); P(S_X), P(S_Y)$

gatecse-2004 operating-system process-synchronization normal 5

Answer key

5.24.27 Process Synchronization: GATE CSE 2006 | Question: 61

The atomic *fetch-and-set* x, y instruction unconditionally sets the memory location x to 1 and fetches the old value of x in y without allowing any intervening access to the memory location x. Consider the following implementation of P and V functions on a binary semaphore S.

```
void P (binary_semaphore *s) {
   unsigned y;
   unsigned *x = &(s->value);
   do {
      fetch-and-set x, y;
   } while (y);
}

void V (binary_semaphore *s) {
   S->value = 0;
}
```

7

Which one of the following is true? 8

A. The implementation may not work if context switching is disabled in P 9
B. Instead of using *fetch-and –set*, a pair of normal load/store can be used
C. The implementation of V is wrong
D. The code does not implement a binary semaphore

gatecse-2006 operating-system process-synchronization normal

Answer key

5.24.28 Process Synchronization: GATE CSE 2006 | Question: 78

Barrier is a synchronization construct where a set of processes synchronizes globally i.e., each process in the set arrives at the barrier and waits for all others to arrive and then all processes leave the barrier. Let the number of processes in the set be three and S be a binary semaphore with the usual P and V functions. Consider the following C implementation of a barrier with line numbers shown on left.

void barrier (void) {

```
1: P(S);
2: process_arrived++;
3: V(S);
```

```
4: while (process_arrived !=3);
5: P(S);
6: process_left++;
7: if (process_left==3) {
8:     process_arrived = 0;
9:     process_left = 0;
10: }
11: V(S);
```

1

}2

The variables *process_arrived* and *process_left* are shared among all processes and are initialized to zero. In a concurrent program all the three processes call the barrier function when they need to synchronize globally. 3

The above implementation of barrier is incorrect. Which one of the following is true? 4

A. The barrier implementation is wrong due to the use of binary semaphore S
B. The barrier implementation may lead to a deadlock if two barrier in invocations are used in immediate succession.
C. Lines 6 to 10 need not be inside a critical section
D. The barrier implementation is correct if there are only two processes instead of three.

5

gatecse-2006 operating-system process-synchronization normal

Answer key

5.24.29 Process Synchronization: GATE CSE 2006 | Question: 79

Barrier is a synchronization construct where a set of processes synchronizes globally i.e., each process in the set arrives at the barrier and waits for all others to arrive and then all processes leave the barrier. Let the number of processes in the set be three and S be a binary semaphore with the usual P and V functions. Consider the following C implementation of a barrier with line numbers shown on left. 6

void barrier (void) { 7

```
1 P(S);
2 process_arrived++;
3 V(S);
4 while (process_arrived !=3);
5   P(S);
6   process_left++;
7   if (process_left==3) {
8     process_arrived = 0;
9     process_left = 0;
10  }
11 V(S);
```

8

}9

The variables process_arrived and process_left are shared among all processes and are initialized to zero. In a concurrent program all the three processes call the barrier function when they need to synchronize globally. 10

Which one of the following rectifies the problem in the implementation? 11

A. Lines 6 to 10 are simply replaced by process_arrived--
B. At the beginning of the barrier the first process to enter the barrier waits until process_arrived becomes zero before proceeding to execute $P(S)$.
C. Context switch is disabled at the beginning of the barrier and re-enabled at the end.
D. The variable process_left is made private instead of shared

12

gatecse-2006 operating-system process-synchronization normal

Answer key

5.24.30 Process Synchronization: GATE CSE 2007 | Question: 58

Two processes, $P1$ and $P2$, need to access a critical section of code. Consider the following synchronization construct used by the processes:

/* P1 */
while (true) {
wants1 = true;
while (wants2 == true);
/* Critical Section */
wants1 = false;
}
/* Remainder section */
	/* P2 */
	while (true) {
	wants2 = true;
	while (wants1 == true);
	/* Critical Section */
	wants2=false;
	}
	/* Remainder section */

1

Here, wants1 and wants2 are shared variables, which are initialized to false. 2
Which one of the following statements is TRUE about the construct?

A. It does not ensure mutual exclusion. 3
B. It does not ensure bounded waiting.
C. It requires that processes enter the critical section in strict alteration.
D. It does not prevent deadlocks, but ensures mutual exclusion.

gatecse-2007 operating-system process-synchronization normal

Answer key

5.24.31 Process Synchronization: GATE CSE 2009 | Question: 33

The **enter_CS()** and **leave_CS()** functions to implement critical section of a process are realized using test-and-set instruction as follows: 4

```
void enter_CS(X)
{
   while(test-and-set(X));
}

void leave_CS(X)
{
   X = 0;
}
```

5

In the above solution, X is a memory location associated with the CS and is initialized to 0. Now consider the following statements: 6

I. The above solution to CS problem is deadlock-free 7
II. The solution is starvation free
III. The processes enter CS in FIFO order
IV. More than one process can enter CS at the same time

Which of the above statements are TRUE? 8

A. (I) only B. (I) and (II) C. (II) and (III) D. (IV) only 9

gatecse-2009 operating-system process-synchronization normal

Answer key

5.24.32 Process Synchronization: GATE CSE 2010 | Question: 23

Consider the methods used by processes $P1$ and $P2$ for accessing their critical sections whenever needed, as given below. The initial values of shared boolean variables $S1$ and $S2$ are randomly assigned.

Method used by P1	Method used by P2
while (S1 == S2); Critical Section S1 = S2;	while (S1 != S2); Critical Section S2 = not(S1);

1

Which one of the following statements describes the properties achieved? 2

A. Mutual exclusion but not progress
B. Progress but not mutual exclusion 3
C. Neither mutual exclusion nor progress
D. Both mutual exclusion and progress

gatecse-2010 operating-system process-synchronization normal 4

Answer key 5

5.24.33 Process Synchronization: GATE CSE 2010 | Question: 45

6

The following program consists of 3 concurrent processes and 3 binary semaphores. The semaphores are initialized as $S0 = 1, S1 = 0$ and $S2 = 0$.

Process P0	Process P1	Process P2

while (true) {
wait (S0);
print '0';
release (S1);
release (S2);
}
	wait (S1);
	release (S0);
		wait (S2);
		release (S0);

7

How many times will process $P0$ print '0'? 8

A. At least twice
B. Exactly twice 9
C. Exactly thrice
D. Exactly once

gatecse-2010 operating-system process-synchronization normal

Answer key

5.24.34 Process Synchronization: GATE CSE 2012 | Question: 32

10

11

Fetch_And_Add(X,i) is an atomic Read-Modify-Write instruction that reads the value of memory location X, increments it by the value i, and returns the old value of X. It is used in the pseudocode shown below to implement a busy-wait lock. L is an unsigned integer shared variable initialized to 0. The value of 0 corresponds to lock being available, while any non-zero value corresponds to the lock being not available.

12

```
AcquireLock(L){
   while (Fetch_And_Add(L,1))
      L = 1;
}

ReleaseLock(L){
   L = 0;
}
```

This implementation 13

A. fails as L can overflow
B. fails as L can take on a non-zero value when the lock is actually available
C. works correctly but may starve some processes
D. works correctly without starvation

14

gatecse-2012 operating-system process-synchronization normal

Answer key

5.24.35 Process Synchronization: GATE CSE 2013 | Question: 34

A shared variable x, initialized to zero, is operated on by four concurrent processes W, X, Y, Z as follows. Each of the processes W and X reads x from memory, increments by one, stores it to memory, and then terminates. Each of the processes Y and Z reads x from memory, decrements by two, stores it to memory, and then terminates. Each process before reading x invokes the P operation (i.e., wait) on a counting semaphore S and invokes the V operation (i.e., signal) on the semaphore S after storing x to memory. Semaphore S is initialized to two. What is the maximum possible value of x after all processes complete execution?

A. -2 B. -1 C. 1 D. 2

gatecse-2013 operating-system process-synchronization normal

Answer key

5.24.36 Process Synchronization: GATE CSE 2013 | Question: 39

A certain computation generates two arrays a and b such that $a[i] = f(i)$ for $0 \leq i < n$ and $b[i] = g(a[i])$ for $0 \leq i < n$. Suppose this computation is decomposed into two concurrent processes X and Y such that X computes the array a and Y computes the array b. The processes employ two binary semaphores R and S, both initialized to zero. The array a is shared by the two processes. The structures of the processes are shown below.

Process X:

```
private i;
for (i=0; i< n; i++) {
 a[i] = f(i);
 ExitX(R, S);
}
```

Process Y:

```
private i;
for (i=0; i< n; i++) {
  EntryY(R, S);
  b[i] = g(a[i]);
}
```

Which one of the following represents the **CORRECT** implementations of ExitX and EntryY?

A.
```
ExitX(R, S) {
 P(R);
 V(S);
}
EntryY(R, S) {
 P(S);
 V(R);
}
```

B.
```
ExitX(R, S) {
 V(R);
 V(S);
}
EntryY(R, S) {
 P(R);
 P(S);
}
```

C.
```
ExitX(R, S) {
 P(S);
 V(R);
}
EntryY(R, S) {
 V(S);
 P(R);
}
```

D.
```
ExitX(R, S) {
 V(R);
 P(S);
}
EntryY(R, S) {
 V(S);
 P(R);
}
```

gatecse-2013 operating-system process-synchronization normal

Answer key

5.24.37 Process Synchronization: GATE CSE 2014 | Set 2 | Question: 31

Consider the procedure below for the *Producer-Consumer* problem which uses semaphores:

```
semaphore n = 0;
semaphore s = 1;
```

```
void producer()
{
   while(true)
   {
      produce();
      semWait(s);
      addToBuffer();
      semSignal(s);
      semSignal(n);
   }
}
```

```
void consumer()
{
   while(true)
   {
      semWait(s);
      semWait(n);
      removeFromBuffer();
      semSignal(s);
      consume();
   }
}
```

Which one of the following is **TRUE**?

A. The producer will be able to add an item to the buffer, but the consumer can never consume it.
B. The consumer will remove no more than one item from the buffer.
C. Deadlock occurs if the consumer succeeds in acquiring semaphore s when the buffer is empty.
D. The starting value for the semaphore n must be 1 and not 0 for deadlock-free operation.

gatecse-2014-set2 operating-system process-synchronization normal

Answer key

5.24.38 Process Synchronization: GATE CSE 2015 | Set 1 | Question: 9

The following two functions $P1$ and $P2$ that share a variable B with an initial value of 2 execute concurrently.

P1() {	P2(){
C = B - 1;	D = 2 * B;
B = 2 * C;	B = D - 1;
}	}

The number of distinct values that B can possibly take after the execution is_____.

gatecse-2015-set1 operating-system process-synchronization normal numerical-answers

Answer key

5.24.39 Process Synchronization: GATE CSE 2015 | Set 3 | Question: 10

Two processes X and Y need to access a critical section. Consider the following synchronization construct used by both the processes

Process X	Process Y

/* other code for process X*/
while (true)
{
varP = true;
while (varQ == true)
{
/* Critical Section */
varP = false;
}
}
/* other code for process X */
	/* other code for process Y */
	while (true)
	{
	varQ = true;
	while (varP == true)
	{
	/* Critical Section */
	varQ = false;
	}
	}
	/* other code for process Y */

1

Here *varP* and *varQ* are shared variables and both are initialized to false. Which one of the following statements is true? 2

A. The proposed solution prevents deadlock but fails to guarantee mutual exclusion
B. The proposed solution guarantees mutual exclusion but fails to prevent deadlock
C. The proposed solution guarantees mutual exclusion and prevents deadlock
D. The proposed solution fails to prevent deadlock and fails to guarantee mutual exclusion

3

gatecse-2015-set3 operating-system process-synchronization normal

Answer key

5.24.40 Process Synchronization: GATE CSE 2016 | Set 1 | Question: 50

Consider the following proposed solution for the critical section problem. There are n processes : $P_0 \ldots P_{n-1}$. In the code, function pmax returns an integer not smaller than any of its arguments .For all $i, t[i]$ is initialized to zero. 4

Code for P_i; 5

```
do {
   c[i]=1; t[i]= pmax (t[0],....,t[n-1])+1; c[i]=0;
   for every j != i in {0,....,n-1} {
      while (c[j]);
      while (t[j] != 0 && t[j] <=t[i]);
   }
   Critical Section;
   t[i]=0;

   Remainder Section;

} while (true);
```

6

Which of the following is TRUE about the above solution? 7

A. At most one process can be in the critical section at any time 8
B. The bounded wait condition is satisfied
C. The progress condition is satisfied
D. It cannot cause a deadlock

gatecse-2016-set1 operating-system process-synchronization difficult ambiguous

Answer key

5.24.41 Process Synchronization: GATE CSE 2016 | Set 2 | Question: 48

Consider the following two-process synchronization solution.

PROCESS 0	Process 1
Entry: loop while (turn == 1); (critical section) Exit: turn = 1;	Entry: loop while (turn == 0); (critical section) Exit turn = 0;

1

The shared variable turn is initialized to zero. Which one of the following is TRUE? 2

A. This is a correct two- process synchronization solution. 3
B. This solution violates mutual exclusion requirement.
C. This solution violates progress requirement.
D. This solution violates bounded wait requirement.

gatecse-2016-set2 operating-system process-synchronization normal

Answer key

5.24.42 Process Synchronization: GATE CSE 2017 | Set 1 | Question: 27

4

A multithreaded program P executes with x number of threads and uses y number of locks for ensuring mutual exclusion while operating on shared memory locations. All locks in the program are *non-reentrant*, i.e., if a thread holds a lock l, then it cannot re-acquire lock l without releasing it. If a thread is unable to acquire a lock, it blocks until the lock becomes available. The *minimum* value of x and the *minimum* value of y together for which execution of P can result in a deadlock are:

A. $x = 1, y = 2$
B. $x = 2, y = 1$ 5
C. $x = 2, y = 2$
D. $x = 1, y = 1$

gatecse-2017-set1 operating-system process-synchronization normal

Answer key

5.24.43 Process Synchronization: GATE CSE 2018 | Question: 40

6

Consider the following solution to the producer-consumer synchronization problem. The shared buffer size is N. Three semaphores $empty$, $full$ and $mutex$ are defined with respective initial values of $0, N$ and 1. Semaphore $empty$ denotes the number of available slots in the buffer, for the consumer to read from. Semaphore $full$ denotes the number of available slots in the buffer, for the producer to write to. The placeholder variables, denoted by P, Q, R and S, in the code below can be assigned either $empty$ or $full$. The valid semaphore operations are: $wait()$ and $sigmal()$. 7

Producer:	Consumer:

do {
wait (P);
wait (mutex);
//Add item to buffer
signal (mutex);
signal (Q);
}while (1);
	do {
	wait (R);
	wait (mutex);
	//consume item from buffer
	signal (mutex);
	signal (S);
	}while (1);

8

Which one of the following assignments tp P, Q, R and S will yield the correct solution? 9

A. $P : full,\ Q : full,\ R : empty,\ S : empty$ 10
B. $P : empty,\ Q : empty,\ R : full,\ S : full$
C. $P : full,\ Q : empty,\ R : empty,\ S : full$
D. $P : empty,\ Q : full,\ R : full,\ S : empty$

gatecse-2018 operating-system process-synchronization normal two-marks

Answer key

5.24.44 Process Synchronization: GATE CSE 2019 | Question: 23

Consider three concurrent processes P_1, P_2 and P_3 as shown below, which access a shared variable D that has been initialized to 100.

P_1	P_2	P_3
:	:	:
:	:	:
$D = D + 20$	$D = D - 50$	$D = D + 10$
:	:	:
:	:	:

The processes are executed on a uniprocessor system running a time-shared operating system. If the minimum and maximum possible values of D after the three processes have completed execution are X and Y respectively, then the value of $Y - X$ is _____

gatecse-2019 numerical-answers operating-system process-synchronization one-mark

Answer key

5.24.45 Process Synchronization: GATE CSE 2024 | Set 2 | Question: 36

Consider a multi-threaded program with two threads $\mathrm{T1}$ and $\mathrm{T2}$. The threads share two semaphores: $s1$ (initialized to 1) and $s2$ (initialized to 0). The threads also share a global variable x (initialized to 0). The threads execute the code shown below.

```
//code of T 1
wait (s1);
x = x+1;
print (x);
wait (s2);
signal(s1);
```

```
// code of T2
wait (s1);
x= x+1;
print (x) ;
signal (s2);
signal (s1);
```

Which of the following outcomes is/are possible when threads $\mathrm{T1}$ and $\mathrm{T2}$ execute concurrently?

A. $\mathrm{T1}$ runs first and prints 1, $\mathrm{T2}$ runs next and prints 2
B. $\mathrm{T2}$ runs first and prints 1, $\mathrm{T1}$ runs next and prints 2
C. $\mathrm{T1}$ runs first and prints 1, $\mathrm{T}\ 2$ does not print anything (deadlock)
D. $\mathrm{T2}$ runs first and prints 1, $\mathrm{T1}$ does not print anything (deadlock)

gatecse-2024-set2 operating-system multiple-selects process-synchronization two-marks

Answer key

5.24.46 Process Synchronization: GATE IT 2004 | Question: 65

The semaphore variables full, empty and mutex are initialized to 0, n and 1, respectively. Process P_1 repeatedly adds one item at a time to a buffer of size n, and process P_2 repeatedly removes one item at a time from the same buffer using the programs given below. In the programs, K, L, M and N are unspecified statements.

P_1

```
while (1) {
    K;
    P(mutex);
    Add an item to the buffer;
    V(mutex);
    L;
}
```

1

P_2

```
while (1) {
    M;
    P(mutex);
    Remove an item from the buffer;
    V(mutex);
    N;
}
```

2

The statements K, L, M and N are respectively 3

A. P(full), V(empty), P(full), V(empty)
B. P(full), V(empty), P(empty), V(full) 4
C. P(empty), V(full), P(empty), V(full)
D. P(empty), V(full), P(full), V(empty)

gateit-2004 operating-system process-synchronization normal 5

Answer key

5.24.47 Process Synchronization: GATE IT 2005 | Question: 41

Given below is a program which when executed spawns two concurrent processes : 6
semaphore $X := 0$;
/* Process now forks into concurrent processes $P1$ & $P2$ */

$P1$	$P2$
repeat forever	repeat forever
$V(X)$;	$P(X)$;
Compute;	Compute;
$P(X)$;	$V(X)$;

7

Consider the following statements about processes $P1$ and $P2$: 8

I. It is possible for process $P1$ to starve. 9
II. It is possible for process $P2$ to starve.

Which of the following holds? 10

A. Both (I) and (II) are true.
B. (I) is true but (II) is false. 11
C. (II) is true but (I) is false
D. Both (I) and (II) are false

gateit-2005 operating-system process-synchronization normal 12

Answer key

5.24.48 Process Synchronization: GATE IT 2005 | Question: 42

Two concurrent processes $P1$ and $P2$ use four shared resources $R1, R2, R3$ and $R4$, as shown below. 13

P1	**P2**
Compute:	Compute;
Use R1;	Use R1;
Use R2;	Use R2;
Use R3;	Use R3;
Use R4;	Use R4;

14

Both processes are started at the same time, and each resource can be accessed by only one process at a time 15
The following scheduling constraints exist between the access of resources by the processes:

- $P2$ must complete use of $R1$ before $P1$ gets access to $R1$. 1
- $P1$ must complete use of $R2$ before $P2$ gets access to $R2$.
- $P2$ must complete use of $R3$ before $P1$ gets access to $R3$.
- $P1$ must complete use of $R4$ before $P2$ gets access to $R4$.

There are no other scheduling constraints between the processes. If only binary semaphores are used to enforce the above scheduling constraints, what is the minimum number of binary semaphores needed? 2

A. 1 B. 2 C. 3 D. 4 3

gateit-2005 operating-system process-synchronization normal 4

Answer key

5.24.49 Process Synchronization: GATE IT 2006 | Question: 55

Consider the solution to the bounded buffer producer/consumer problem by using general semaphores $S, F,$ and E. The semaphore S is the mutual exclusion semaphore initialized to 1. The semaphore F corresponds to the number of free slots in the buffer and is initialized to N. The semaphore E corresponds to the number of elements in the buffer and is initialized to 0. 5

Producer Process	Consumer Process
Produce an item;	Wait(E);
Wait(F);	Wait(S);
Wait(S);	Remove an item from the buffer;
Append the item to the buffer;	Signal(S);
Signal(S);	Signal(F);
Signal(E);	Consume the item;

6

Which of the following interchange operations may result in a deadlock? 7

I. Interchanging Wait (F) and Wait (S) in the Producer process 8
II. Interchanging Signal (S) and Signal (F) in the Consumer process

A. (I) only B. (II) only 9
C. Neither (I) nor (II) D. Both (I) and (II)

gateit-2006 operating-system process-synchronization normal 10

Answer key

5.24.50 Process Synchronization: GATE IT 2007 | Question: 10

Processes $P1$ and $P2$ use critical_flag in the following routine to achieve mutual exclusion. Assume that critical_flag is initialized to FALSE in the main program. 11

```
get_exclusive_access ( )
{
   if (critical _flag == FALSE) {
      critical_flag = TRUE ;
      critical_region () ;
      critical_flag = FALSE;
   }
}
```

12

Consider the following statements. 13

i. It is possible for both $P1$ and $P2$ to access critical_region concurrently. 14
ii. This may lead to a deadlock.

Which of the following holds? 15

A. (i) is false (ii) is true B. Both (i) and (ii) are false 16
C. (i) is true (ii) is false D. Both (i) and (ii) are true

gateit-2007 operating-system process-synchronization normal 17

Answer key

5.24.51 Process Synchronization: GATE IT 2007 | Question: 56

Synchronization in the classical readers and writers problem can be achieved through use of semaphores. In the following incomplete code for readers-writers problem, two binary semaphores mutex and wrt are used to obtain synchronization

```
wait (wrt)
writing is performed
signal (wrt)
wait (mutex)
readcount = readcount + 1
if readcount = 1 then S1
S2
reading is performed
S3
readcount = readcount - 1
if readcount = 0 then S4
signal (mutex)
```

The values of $S1, S2, S3, S4,$ (in that order) are

A. signal (mutex), wait (wrt), signal (wrt), wait (mutex)
B. signal (wrt), signal (mutex), wait (mutex), wait (wrt)
C. wait (wrt), signal (mutex), wait (mutex), signal (wrt)
D. signal (mutex), wait (mutex), signal (mutex), wait (mutex)

gateit-2007 operating-system process-synchronization normal

Answer key

5.24.52 Process Synchronization: GATE IT 2008 | Question: 53

The following is a code with two threads, producer and consumer, that can run in parallel. Further, S and Q are binary semaphores quipped with the standard P and V operations.

```
semaphore S = 1, Q = 0;
integer x;

producer:                consumer:
while (true) do           while (true) do
   P(S);                   P(Q);
   x = produce ();             consume (x);
   V(Q);                   V(S);
done                     done
```

Which of the following is TRUE about the program above?

A. The process can deadlock
B. One of the threads can starve
C. Some of the items produced by the producer may be lost
D. Values generated and stored in 'x' by the producer will always be consumed before the producer can generate a new value

gateit-2008 operating-system process-synchronization normal

Answer key

5.25 Resource Allocation (27)

Practice Tests: Test 1 (15Q) Test 2 (10Q)

5.25.1 Resource Allocation: GATE CSE 1988 | Question: 11

A number of processes could be in a deadlock state if none of them can execute due to non-availability of sufficient resources. Let $P_i, 0 \leq i \leq 4$ represent five processes and let there be four resources types $r_j, 0 \leq j \leq 3$. Suppose the following data structures have been used.

Available: A vector of length 4 such that if Available $[i] = k$, there are k instances of resource type r_j available in

the system.1

Allocation. A 5×4 matrix defining the number of each type currently allocated to each process. If Allocation $[i, j] = k$ then process p_i is currently allocated k instances of resource type r_j.2

Max. A 5×4 matrix indicating the maximum resource need of each process. If $Max[i, j] = k$ then process p_i, may need a maximum of k instances of resource type r_j in order to complete the task.3

Assume that system allocated resources only when it does not lead into an unsafe state such that resource requirements in future never cause a deadlock state. Now consider the following snapshot of the system.4

Allocation

	r_0	r_1	r_2	r_3
p_0	0	0	1	2
p_1	1	0	0	0
p_2	1	3	5	4
p_3	0	6	3	2
p_4	0	0	1	4

Max

r_0	r_1	r_2	r_3
0	0	1	2
1	7	5	0
2	3	5	6
0	6	5	2
0	6	5	6

Available

r_0	r_1	r_2	r_3
1	5	2	0

5

Is the system currently in a safe state? If yes, explain why.6

gate1988 normal descriptive operating-system resource-allocation

Answer key

5.25.2 Resource Allocation: GATE CSE 1989 | Question: 11a

i. A system of four concurrent processes, P, Q, R and S, use shared resources A, B and C. The sequences in which processes, P, Q, R and S request and release resources are as follows:7

Process P:	1.	P requests A
	2.	P requests B
	3.	P releases A
	4.	P releases B
Process Q:	1.	Q requests C
	2.	Q requests A
	3.	Q releases C
	4.	P releases A
Process R:	1.	R requests B
	2.	R requests C
	3.	R releases B
	4.	R releases C
Process S:	1.	S requests A
	2.	S requests C
	3.	S releases A
	4.	S releases C

8

If a resource is free, it is granted to a requesting process immediately. There is no preemption of granted resources. A resource is taken back from a process only when the process explicitly releases it.9

Can the system of four processes get into a deadlock? If yes, give a sequence (ordering) of operations (for requesting and releasing resources) of these processes which leads to a deadlock.10

ii. Will the processes always get into a deadlock? If your answer is no, give a sequence of these operations which leads to completion of all processes.11
iii. What strategies can be used to prevent deadlocks in a system of concurrent processes using shared resources

if preemption of granted resources is not allowed? 1

descriptive gate1989 operating-system resource-allocation

Answer key

5.25.3 Resource Allocation: GATE CSE 1992 | Question: 02-xi

A computer system has 6 tape devices, with n processes competing for them. Each process may need 3 tape drives. The maximum value of n for which the system is guaranteed to be deadlock-free is: 2

A. 2 B. 3 C. 4 D. 1 3

gate1992 operating-system resource-allocation normal multiple-selects 4

Answer key 5

5.25.4 Resource Allocation: GATE CSE 1993 | Question: 7.9, UGCNET-Dec2012-III: 41 6

Consider a system having m resources of the same type. These resources are shared by 3 processes A, B, and C which have peak demands of $3, 4$, and 6 respectively. For what value of m deadlock will not occur? 7

A. 7 B. 9 C. 10 D. 13 8

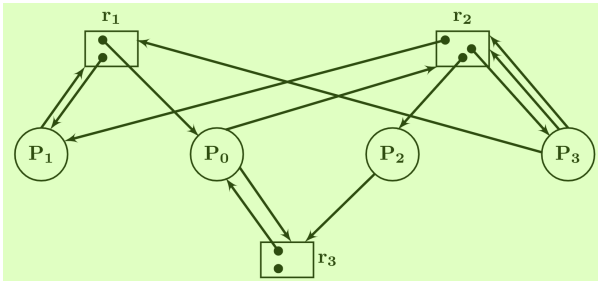
E. 15

gate1993 operating-system resource-allocation normal ugcnetcse-dec2012-paper3 multiple-selects 9

Answer key

5.25.5 Resource Allocation: GATE CSE 1994 | Question: 28

Consider the resource allocation graph in the figure. 10



11

A. Find if the system is in a deadlock state 12
B. Otherwise, find a safe sequence

gate1994 operating-system resource-allocation normal descriptive

Answer key 13

5.25.6 Resource Allocation: GATE CSE 1996 | Question: 22

A computer system uses the Banker's Algorithm to deal with deadlocks. Its current state is shown in the table below, where $P0$, $P1$, $P2$ are processes, and $R0$, $R1$, $R2$ are resources types.

Maximum Need

	$R0$	$R1$	$R2$
$P0$	4	1	2
$P1$	1	5	1
$P2$	1	2	3

Current Allocation

	$R0$	$R1$	$R2$
$P0$	1	0	2
$P1$	0	3	1
$P2$	1	0	2

Available

$R0$	$R1$	$R2$
2	2	0

A. Show that the system can be in this state
B. What will the system do on a request by process $P0$ for one unit of resource type $R1$?

gate1996 operating-system resource-allocation normal descriptive

Answer key

5.25.7 Resource Allocation: GATE CSE 1997 | Question: 6.7

An operating system contains 3 user processes each requiring 2 units of resource R. The minimum number of units of R such that no deadlocks will ever arise is

A. 3 B. 5 C. 4 D. 6

gate1997 operating-system resource-allocation normal

Answer key

5.25.8 Resource Allocation: GATE CSE 1997 | Question: 75

An operating system handles requests to resources as follows.

A process (which asks for some resources, uses them for some time and then exits the system) is assigned a unique timestamp are when it starts. The timestamps are monotonically increasing with time. Let us denote the timestamp of a process P by $TS(P)$.

When a process P requests for a resource the OS does the following:

i. If no other process is currently holding the resource, the OS awards the resource to P.
ii. If some process Q with $TS(Q) < TS(P)$ is holding the resource, the OS makes P wait for the resources.
iii. If some process Q with $TS(Q) > TS(P)$ is holding the resource, the OS restarts Q and awards the resources to P. (Restarting means taking back the resources held by a process, killing it and starting it again with the same timestamp)

When a process releases a resource, the process with the smallest timestamp (if any) amongst those waiting for the resource is awarded the resource.

A. Can a deadlock over arise? If yes, show how. If not prove it.
B. Can a process P ever starve? If yes, show how. If not prove it.

gate1997 operating-system resource-allocation normal descriptive

Answer key

5.25.9 Resource Allocation: GATE CSE 1998 | Question: 1.32

A computer has six tape drives, with n processes competing for them. Each process may need two drives. What is the maximum value of n for the system to be deadlock free?

A. 6 B. 5 C. 4 D. 3

gate1998 operating-system resource-allocation normal

Answer key

5.25.10 Resource Allocation: GATE CSE 2000 | Question: 2.23

Which of the following is not a valid deadlock prevention scheme?

A. Release all resources before requesting a new resource.
B. Number the resources uniquely and never request a lower numbered resource than the last one requested.
C. Never request a resource after releasing any resource.
D. Request and all required resources be allocated before execution.

gatecse-2000 operating-system resource-allocation normal

Answer key

5.25.11 Resource Allocation: GATE CSE 2001 | Question: 19

Two concurrent processes $P1$ and $P2$ want to use resources $R1$ and $R2$ in a mutually exclusive manner. Initially, $R1$ and $R2$ are free. The programs executed by the two processes are given below.

	Program for P1:		**Program for P2:**
S1:	While ($R1$ is busy) do no-op;	Q1:	While ($R1$ is busy) do no-op;
S2:	Set $R1 \leftarrow$ busy;	Q2:	Set $R1 \leftarrow$ busy;
S3:	While ($R2$ is busy) do no-op;	Q3:	While ($R2$ is busy) do no-op;
S4:	Set $R2 \leftarrow$ busy;	Q4:	Set $R2 \leftarrow$ busy;
S5:	Use $R1$ and $R2$;	Q5:	Use $R1$ and $R2$;
S6:	Set $R1 \leftarrow$ free;	Q6:	Set $R2 \leftarrow$ free;
S7:	Set $R2 \leftarrow$ free;	Q7:	Set $R1 \leftarrow$ free;

A. Is mutual exclusion guaranteed for $R1$ and $R2$? If not show a possible interleaving of the statements of $P1$ and $P2$ such mutual exclusion is violated (i.e., both $P1$ and $P2$ use $R1$ and $R2$ at the same time).
B. Can deadlock occur in the above program? If yes, show a possible interleaving of the statements of $P1$ and $P2$ leading to deadlock.
C. Exchange the statements $Q1$ and $Q3$ and statements $Q2$ and $Q4$. Is mutual exclusion guaranteed now? Can deadlock occur?

gatecse-2001 operating-system resource-allocation normal descriptive

Answer key

5.25.12 Resource Allocation: GATE CSE 2005 | Question: 71

Suppose n processes, $P_1, \ldots P_n$ share m identical resource units, which can be reserved and released one at a time. The maximum resource requirement of process P_i is s_i, where $s_i > 0$. Which one of the following is a sufficient condition for ensuring that deadlock does not occur?

A. $\forall i,\ s_i, < m$
B. $\forall i,\ s_i < n$
C. $\sum_{i=1}^{n} s_i < (m+n)$
D. $\sum_{i=1}^{n} s_i < (m \times n)$

gatecse-2005 operating-system resource-allocation normal

Answer key

5.25.13 Resource Allocation: GATE CSE 2006 | Question: 66

Consider the following snapshot of a system running n processes. Process i is holding x_i instances of a resource R, $1 \leq i \leq n$. Currently, all instances of R are occupied. Further, for all i, process i has placed a request for an additional y_i instances while holding the x_i instances it already has. There are exactly two processes p and q and such that $y_p = y_q = 0$. Which one of the following can serve as a necessary condition to guarantee that the system is not approaching a deadlock?

A. $\min(x_p, x_q) < \max_{k \neq p,q} y_k$
B. $x_p + x_q \geq \min_{k \neq p,q} y_k$
C. $\max(x_p, x_q) > 1$
D. $\min(x_p, x_q) > 1$

gatecse-2006 operating-system resource-allocation normal

Answer key

5.25.14 Resource Allocation: GATE CSE 2007 | Question: 57

A single processor system has three resource types X, Y and Z, which are shared by three processes. There are 5 units of each resource type. Consider the following scenario, where the column **alloc** denotes the number of units of each resource type allocated to each process, and the column **request** denotes the number of units of each resource type requested by a process in order to complete execution. Which of these processes will finish **LAST**?

	alloc			request		
	X	Y	Z	X	Y	Z
P0	1	2	1	1	0	3
P1	2	0	1	0	1	2
P2	2	2	1	1	2	0

A. $P0$
B. $P1$
C. $P2$
D. None of the above, since the system is in a deadlock

gatecse-2007 operating-system resource-allocation normal

Answer key

5.25.15 Resource Allocation: GATE CSE 2008 | Question: 65

Which of the following is NOT true of deadlock prevention and deadlock avoidance schemes?

A. In deadlock prevention, the request for resources is always granted if the resulting state is safe
B. In deadlock avoidance, the request for resources is always granted if the resulting state is safe
C. Deadlock avoidance is less restrictive than deadlock prevention
D. Deadlock avoidance requires knowledge of resource requirements *apriori*..

gatecse-2008 operating-system easy resource-allocation

Answer key

5.25.16 Resource Allocation: GATE CSE 2009 | Question: 30

Consider a system with 4 types of resources $R1$ (3 units), $R2$ (2 units), $R3$ (3 units), $R4$ (2 units). A non-preemptive resource allocation policy is used. At any given instance, a request is not entertained if it cannot be completely satisfied. Three processes $P1$, $P2$, $P3$ request the resources as follows if executed independently.

Process P1:	**Process P2:**	**Process P3:**
$t = 0$: requests 2 units of $R2$	$t = 0$: requests 2 units of $R3$	$t = 0$: requests 1 unit of $R4$
$t = 1$: requests 1 unit of $R3$	$t = 2$: requests 1 unit of $R4$	$t = 2$: requests 2 units of $R1$
$t = 3$: requests 2 units of $R1$	$t = 4$: requests 1 unit of $R1$	$t = 5$: releases 2 units of $R1$
$t = 5$: releases 1 unit of $R2$ and 1 unit of $R1$	$t = 6$: releases 1 unit of $R3$	$t = 7$: requests 1 unit of $R2$
$t = 7$: releases 1 unit of $R3$	$t = 8$: Finishes	$t = 8$: requests 1 unit of $R3$
$t = 8$: requests 2 units of $R4$		$t = 9$: Finishes
$t = 10$: Finishes		

Which one of the following statements is TRUE if all three processes run concurrently starting at time $t = 0$?

A. All processes will finish without any deadlock
B. Only $P1$ and $P2$ will be in deadlock
C. Only $P1$ and $P3$ will be in deadlock
D. All three processes will be in deadlock

gatecse-2009 operating-system resource-allocation normal

Answer key

5.25.17 Resource Allocation: GATE CSE 2010 | Question: 46

A system has n resources $R_0, \ldots, R_{n-1}$, and k processes $P_0, \ldots, P_{k-1}$. The implementation of the resource request logic of each process P_i is as follows:

```
if(i%2 == 0){
   if(i < n) request R_i;
   if(i+2 < n) request R_{i+2};}
else{
   if(i < n) request R_{n-i};
   if(i+2 < n) request R_{n-i-2};}
```

In which of the following situations is a deadlock possible?

A. $n = 40, k = 26$ B. $n = 21, k = 12$ C. $n = 20, k = 10$ D. $n = 41, k = 19$

gatecse-2010 operating-system resource-allocation normal

Answer key

5.25.18 Resource Allocation: GATE CSE 2013 | Question: 16

Three concurrent processes X, Y, and Z execute three different code segments that access and update certain shared variables. Process X executes the P operation (i.e., *wait*) on semaphores a, b, and c; process Y executes the P operation on semaphores b, c, and d; process Z executes the P operation on semaphores c, d, and a before entering the respective code segments. After completing the execution of its code segment, each process invokes the V operation (i.e., *signal*) on its three semaphores. All semaphores are binary semaphores initialized to one. Which one of the following represents a deadlock-free order of invoking the P operations by the processes?

A. $X : P(a)P(b)P(c)\ Y : P(b)P(c)P(d)\ Z : P(c)P(d)P(a)$
B. $X : P(b)P(a)P(c)\ Y : P(b)P(c)P(d)\ Z : P(a)P(c)P(d)$
C. $X : P(b)P(a)P(c)\ Y : P(c)P(b)P(d)\ Z : P(a)P(c)P(d)$
D. $X : P(a)P(b)P(c)\ Y : P(c)P(b)P(d)\ Z : P(c)P(d)P(a)$

gatecse-2013 operating-system resource-allocation normal

Answer key

5.25.19 Resource Allocation: GATE CSE 2014 | Set 1 | Question: 31

An operating system uses the *Banker's algorithm* for deadlock avoidance when managing the allocation of three resource types $X, Y,$ and Z to three processes $P0, P1,$ and $P2$. The table given below presents the current system state. Here, the *Allocation matrix* shows the current number of resources of each type allocated to each process and the *Max matrix* shows the maximum number of resources of each type required by each process during its execution.

	Allocation			Max		
	X	Y	Z	X	Y	Z
P0	0	0	1	8	4	3
P1	3	2	0	6	2	0
P2	2	1	1	3	3	3

There are 3 units of type X, 2 units of type Y and 2 units of type Z still available. The system is currently in a **safe** state. Consider the following independent requests for additional resources in the current state:

REQ1: $P0$ requests 0 units of X, 0 units of Y and 2 units of Z

REQ2: $P1$ requests 2 units of X, 0 units of Y and 0 units of Z

Which one of the following is **TRUE**?

A. Only REQ1 can be permitted.
B. Only REQ2 can be permitted.
C. Both REQ1 and REQ2 can be
D. Neither REQ1 nor REQ2 can be

permitted.

permitted.1

gatecse-2014-set1 operating-system resource-allocation normal

Answer key

5.25.20 Resource Allocation: GATE CSE 2014 | Set 3 | Question: 31

A system contains three programs and each requires three tape units for its operation. The minimum2 number of tape units which the system must have such that deadlocks never arise is _____.

gatecse-2014-set3 operating-system resource-allocation numerical-answers easy 3

Answer key

5.25.21 Resource Allocation: GATE CSE 2015 | Set 2 | Question: 23

4

A system has 6 identical resources and N processes competing for them. Each process can request at5 most 2 resources. Which one of the following values of N could lead to a deadlock?

A. 1 B. 2 C. 3 D. 4 6

gatecse-2015-set2 operating-system resource-allocation easy 7

Answer key

5.25.22 Resource Allocation: GATE CSE 2015 | Set 3 | Question: 52

8

Consider the following policies for preventing deadlock in a system with mutually exclusive resources. 9

I. Process should acquire all their resources at the beginning of execution. If any resource is not available, all resources acquired so far are released. 10
II. The resources are numbered uniquely, and processes are allowed to request for resources only in increasing resource numbers
III. The resources are numbered uniquely, and processes are allowed to request for resources only in deccreasing resource numbers
IV. The resources are numbered uniquely. A processes is allowed to request for resources only for a resource with resource number larger than its currently held resources

Which of the above policies can be used for preventing deadlock? 11

A. Any one of (I) and (III) but not (II) or (IV)
B. Any one of (I), (III) and (IV) but not (II) 12
C. Any one of (II) and (III) but not (I) or (IV)
D. Any one of (I), (II), (III) and (IV)

gatecse-2015-set3 operating-system resource-allocation normal

Answer key

5.25.23 Resource Allocation: GATE CSE 2017 | Set 2 | Question: 33

13

A system shares 9 tape drives. The current allocation and maximum requirement of tape drives for that processes are shown below:

Process	Current Allocation	Maximum Requirement
P1	3	7
P2	1	6
P3	3	5

14

Which of the following best describes current state of the system? 15

A. Safe, Deadlocked
B. Safe, Not Deadlocked 16
C. Not Safe, Deadlocked
D. Not Safe, Not Deadlocked

gatecse-2017-set2 operating-system resource-allocation normal

Answer key

5.25.24 Resource Allocation: GATE CSE 2019 | Question: 39

Consider the following snapshot of a system running n concurrent processes. Process i is holding X_i instances of a resource R, $1 \leq i \leq n$. Assume that all instances of R are currently in use. Further, for all i, process i can place a request for at most Y_i additional instances of R while holding the X_i instances it already has. Of the n processes, there are exactly two processes p and q such that $Y_p = Y_q = 0$. Which one of the following conditions guarantees that no other process apart from p and q can complete execution?

A. $X_p + X_q < \text{Min}\{Y_k \mid 1 \leq k \leq n, k \neq p, k \neq q\}$
B. $X_p + X_q < \text{Max}\{Y_k \mid 1 \leq k \leq n, k \neq p, k \neq q\}$
C. $\text{Min}(X_p, X_q) \geq \text{Min}\{Y_k \mid 1 \leq k \leq n, k \neq p, k \neq q\}$
D. $\text{Min}(X_p, X_q) \leq \text{Max}\{Y_k \mid 1 \leq k \leq n, k \neq p, k \neq q\}$

gatecse-2019 operating-system two-marks resource-allocation

Answer key

5.25.25 Resource Allocation: GATE CSE 2022 | Question: 16

Which of the following statements is/are TRUE with respect to deadlocks?

A. Circular wait is a necessary condition for the formation of deadlock.
B. In a system where each resource has more than one instance, a cycle in its wait-for graph indicates the presence of a deadlock.
C. If the current allocation of resources to processes leads the system to unsafe state, then deadlock will necessarily occur.
D. In the resource-allocation graph of a system, if every edge is an assignment edge, then the system is not in deadlock state.

gatecse-2022 operating-system resource-allocation multiple-selects one-mark

Answer key

5.25.26 Resource Allocation: GATE IT 2005 | Question: 62

Two shared resources R_1 and R_2 are used by processes P_1 and P_2. Each process has a certain priority for accessing each resource. Let T_{ij} denote the priority of P_i for accessing R_j. A process P_i can snatch a resource R_k from process P_j if T_{ik} is greater than T_{jk}.
Given the following :

I. $T_{11} > T_{21}$
II. $T_{12} > T_{22}$
III. $T_{11} < T_{21}$
IV. $T_{12} < T_{22}$

Which of the following conditions ensures that P_1 and P_2 can never deadlock?

A. (I) and (IV) B. (II) and (III) C. (I) and (II) D. None of the above

gateit-2005 operating-system resource-allocation normal

Answer key

5.25.27 Resource Allocation: GATE IT 2008 | Question: 54

An operating system implements a policy that requires a process to release all resources before making a request for another resource. Select the TRUE statement from the following:

A. Both starvation and deadlock can occur
B. Starvation can occur but deadlock cannot occur
C. Starvation cannot occur but deadlock can occur
D. Neither starvation nor deadlock can occur

gateit-2008 operating-system resource-allocation normal